

Раздел 2. Основные концепции языков программирования (Java)

Содержание

1. Фундаментальные типы данных. Целочисленные типы, представление отрицательных чисел. Числа с плавающей точкой, их двоичное представление. Символьный и логический типы. Явные и неявные приведения типов. Перегрузка функций.
2. Классы. Объекты. Поля, методы. Конструкторы. Ключевое слово `this`. Статические члены класса. Модификаторы доступа, инкапсуляция. Внутренние классы, локальные классы, их особенности и примеры.
3. Наследование. Преобразования типов между родителем и наследником. Переопределение методов у наследника. Абстрактные классы. Интерфейсы. Принцип подстановки Лисков.
4. Исключения. Иерархия исключений. Проверяемые и непроверяемые исключения. Конструкция `try with resources`.
5. Стандартные коллекции: `ArrayList`, `LinkedList`, `HashMap`, `TreeMap`, `LinkedHashMap`. Их основные методы и внутреннее устройство.
6. Лямбда-функции. Элементы функционального программирования. Захват окружения. Ссылки на методы.
7. `Optional`. `Stream API`. Промежуточные и терминальные операции. Коллекторы, комбинации коллекторов.
8. Аннотации. Объявление и применение аннотаций. Мета-аннотации `@Target`, `@Retention`. Параметры аннотаций.
9. Основы многопоточности. Потoki (threads). Создание и завершение потоков. Проблема `race condition`. Мьютексы и их использование.
10. `Spring Framework`. Чем отличается управление объектами через `'new'` от управления через `Inversion of Control`. Какие способы внедрения зависимостей поддерживаются. Что такое `Spring Bean` и какие аннотации используются для создания бинов. Скоупы бинов.
11. `Spring MVC`. Отличие `@Controller` от `@RestController`. Маппинг HTTP-запросов (`@GetMapping`, `@PostMapping` и т.д.). Обработка исключений с помощью `@ControllerAdvice`. Интерсепторы. Валидация объектов (`@NotBlank`, `@Email` и т.д.).
12. `OpenAPI`. Использование `OpenAPI` в документирование REST API. Инструменты для генерации документации `OpenAPI` в `Spring`.
13. `Hibernate`. Как `Hibernate` связан с JPA. Как работают `@OneToMany`, `@ManyToOne`, `@ManyToMany`? В чем разница между `EAGER` и `LAZY` загрузкой? Жизненный цикл

сущности в Hibernate. Транзакции в Spring (`@Transactional`), параметры `isolation` и `propagation` .

14. Spring Security. Настройка аутентификации через базу данных. Реализация JWT-аутентификации в Spring Security.
15. Синхронное и асинхронное взаимодействие в распределенной среде, их преимущества и недостатки. Реализации инструментов (HTTP, брокеры сообщений и т.д).

1. Фундаментальные типы данных. Целочисленные типы, представление отрицательных чисел. Числа с плавающей точкой, их двоичное представление. Символьный и логический типы. Явные и неявные приведения типов. Перегрузка функций.

Все значения в Java делятся на два типа: примитивы и объекты. К примитивам относятся следующие типы данных:

- `byte` - 8-битовое целое число со знаком. Может принимать значение от -128 до 127 .
- `short` - 16-битовое целое число со знаком. Может принимать значение от -32768 до 32767 .
- `int` - 32-битовое целое число со знаком. Может принимать значение от -2^{31} до $2^{31} - 1$.
- `long` - 64-битовое целое число. Число со знаком может принимать значение от -2^{63} до $2^{63} - 1$.
- `float` - 32-битовое число с плавающей запятой.
- `double` - 64-битовое число с плавающей запятой.
- `boolean` - логический тип данных, может иметь только 2 значения, `true` или `false` .
- `char` - один символ в формате Unicode.

Любые другие значения являются объектами.

В Java переменные хранятся следующим образом:

1. Локальные переменные, такие как примитивы и ссылки на объекты, создаются в стеке.
2. Объекты – в куче (heap).

Java – язык *pass by value*, то есть значения переменной копируются при передаче в функцию. В случае примитивов копируется непосредственно значение, для объектов –

копия ссылки.

Целочисленные типы

`byte` (1 байт), `short` (2 байт), `int` (4 байт) и `long` (8 байт).

`byte` в основном используется для уменьшения расхода памяти при хранении данных в массивах (например, хранение данных, полученных по сети в некотором буфере, который будет реализован в виде массива байт). В арифметических операциях он не используется. Связано это с реализацией Java Virtual Machine (JVM). Так как большинство систем 32 или 64 разрядные, то `byte` и `short` при вычислениях будут приведены к 32-битному `int`:

```
byte a = 1;
byte b = 2;
byte result = a + b;
```

```
error: incompatible types: possible lossy conversion from int to byte
```

`long` позволяет считать большие числа. Часто используется при работе со временем (можно встретить как тип для аргумента `millis` при работе с потоками). Для обозначения того, что число это `long` после числа ставят литерал `L`: `long n = 10L`; `\\` можно `10L`

"Минус" для отрицательных чисел берётся из старшего бита, т.е. 1 это запись -128 , а 1111111 – запись 127.

Числа с плавающей точкой

`float` и `double`, литералы `1.1F` и `1.1` (или `1.1D`) соответственно. Они реализуют международный стандарт IEEE 754: числовые типы данных с плавающей точкой состоят из трёх частей: *знака*, *мантиссы* и *порядка*: $(-1)^2$.

- **float**: 4 байта, точность ~7 десятичных знаков. Хранится как: 1 бит на знак, 8 на порядок, 23 на мантиссу.
- **double**: 8 байт, точность ~15–17 десятичных знаков. Хранится как: 1 бит на знак, 11 на порядок, 52 на мантиссу.

Особенности:

- Можно также использовать научную нотация для литералов: `1.1e-3`
- Из-за такой формы хранения существует два нуля: `.` и `-`.
- Специальные значения: *nini* и $-nini$ (деление на ноль полож./отриц. числа соответственно), а также *NaN* (Not a Number, ноль на ноль, sqrt из отрицательного

числа, сложение + и - бесконечностей и т.п.).

Символьный и логический типы.

`boolean` может хранить только два значения: `true` (истина) или `false` (ложь). Размер неизвестен и зависит от JVM (Например может быть: 8 бит в массивах `boolean[]` на куче, а на стеке он хранится как 32-битный `int`).

`char` занимает в памяти 16 бит и описывает символ. В Java для `char` используется кодировка Unicode. Символ можно задать в соответствии с двумя таблицами: таблица Unicode символов и таблица ASCII символов. Для литералов используются одинарные кавычки.

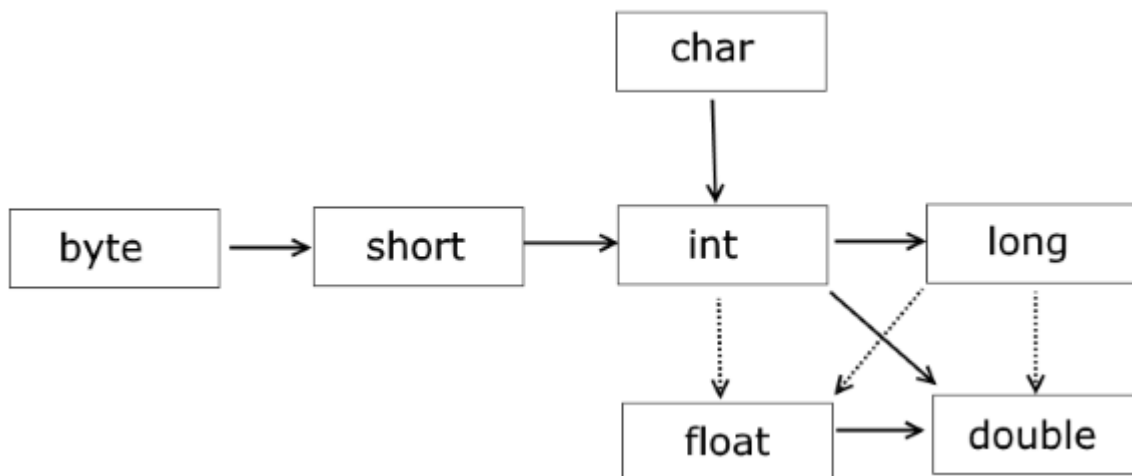
```
char symbolF = 'f';
symbolF = 102;
symbolF = '\u0066';
```

Также является числом и поддерживает математические операции (в которых кастуется к `int`).

Явные и неявные преобразования типов

Каждый примитивный тип в Java имеет фиксированный размер, что определяет его диапазон и точность. При взаимодействии типов с разной разрядностью возникают ситуации несоответствия, которые Java разрешает либо автоматически, либо требует вмешательства программиста (явное приведение).

Автоматические преобразования (неявные, расширение) выполняются по следующим цепочкам:



Стрелками на рисунке показано, какие преобразования типов могут выполняться

автоматически. Пунктирными стрелками показаны автоматические преобразования с потерей точности.

Автоматически без каких-либо проблем производятся расширяющие преобразования (widening) – они расширяют представление объекта в памяти. Например:

```
byte b = 4;  
int i = b;
```

Некоторые преобразования могут производиться автоматически между типами данных одинаковой разрядности или даже от типа данных с большей разрядностью к типу с меньшей разрядностью. Это следующие цепочки преобразований: `int -> float`, `long -> float` и `long -> double`. Они производятся без ошибок, но при преобразовании мы можем столкнуться с потерей информации.

```
int a = 2147483647;  
float b = a; // int -> float  
System.out.println(b); // 2.1474836E9
```

Внутри арифметических выражений происходит *продвижение типов*. Java всегда подтягивает аргументы к "старшему" типу перед вычислением:

- Если есть хоть один `double`, всё станет `double`.
- Иначе, если есть `float`, всё станет `float`.
- Иначе, если есть `long`, всё станет `long`.
- `byte`, `short` и `char` изначально преобразуются в `int`.

Во всех остальных преобразованиях примитивных типов явным образом применяется операция преобразования типов. Обычно это сужающие преобразования (narrowing) от типа с большей разрядностью к типу с меньшей разрядностью:

```
long a = 41;  
int b = (int) a;
```

При применении явных преобразований мы можем столкнуться с потерей данных. Например, в следующем коде у нас не возникнет никаких проблем:

```
int a = 5;  
byte b = (byte) a;
```

Число 5 вполне укладывается в диапазон значений типа `byte`, поэтому после преобразования переменная `b` будет равна 5. Однако в другом примере:

```
int a = 258
byte b = (byte) a;
```

результатом будет число 2. В данном случае число 258 вне диапазона для типа `byte` (от -128 до 127), поэтому произойдет усечение значения. Число `a`, которое равно 258, в двоичном системе будет равно `00000000 00000000 00000001 00000010`. Значения типа `byte` занимают в памяти только 8 бит. Поэтому двоичное представление числа `int` усекается до 8 правых разрядов, то есть `00000010`, что в десятичной системе дает число 2.

Другие подобные случаи:

```
int a = 130;
byte b = (byte) a;
System.out.println(b);

a = 128;
b = (byte) a;
System.out.println(b);

a = 256;
b = (byte) a;
System.out.println(b);
```

```
-126
-128
0
```

Перегрузка функций

Перегрузка – это возможность определить в одном классе несколько методов с одинаковым именем, но разными входными параметрами. Это проявление статического полиморфизма (того, что разрешается во время компиляции).

```
public class Program {
    public static void main(String[] args) {
        Calcualtor calc = new Calcualtor();
        System.out.println(calc.sum(2, 3));
        System.out.println(calc.sum(2.3, 3.2));
        System.out.println(calc.sum(2.3, 3.2, 4));
    }
}
```

```

    }
}

class Calcualtor {
    int sum(int x, int y){
        return x + y;
    }

    double sum(double x, double y){
        return x + y;
    }

    double sum(double x, double y, long z){
        return x + y + z;
    }
}

```

На перегрузку методов влияют количество и типы параметров. Однако различие в типе возвращаемого значения для перегрузки не имеют никакого значения. Например, в следующем случае методы sum различаются по типу возвращаемого значения:

```

public class Program{
    public static void main(String[] args){
        Calcualtor calc = new Calcualtor();
        System.out.println(calc.sum(2, 3));
        System.out.println(calc.sum(2, 3));
    }
}

class Calcualtor {
    int sum(int x, int y){
        return x + y;
    }

    // error: method sum(int,int) is already defined in class Calcualtor
    double sum(int x, int y){
        return x + y;
    }
}

```

2. Классы. Объекты. Поля, методы. Конструкторы. Ключевое слово this. Статические члены класса.

Модификаторы доступа, инкапсуляция. Внутренние классы, локальные классы, их особенности и примеры.

Java является объектно-ориентированным языком, поэтому такие понятия как "класс" и "объект" играют в нем ключевую роль. Любую программу на Java можно представить как набор взаимодействующих между собой объектов. Шаблоном или описанием объекта является класс, а объект представляет экземпляр этого класса.

Класс определяется с помощью ключевого слова `class`, после которого идет название класса:

```
class MyAwesomeClass {  
    \\todo  
}
```

Объект создается с помощью оператора `new`. В этот момент на куче выделяется память.

```
MyAwesomeClass mac = new MyAwesomeClass();
```

Любой объект может обладать двумя основными характеристиками: состояние - некоторые данные, которые хранит объект, и поведение - действия, которые может совершать объект. Для хранения состояния объекта в классе применяются поля (переменные класса), поведение определяется методами:

```
class Person{  
    String name;  
    int age;  
  
    void print(){  
        System.out.printf("Name: %s \tAge: %d\n", name, age);  
    }  
}
```

Доступ к ним вне класса получаем через точку:

```
объект.полеКласса  
объект.методКласса(параметры метода)
```

Конструкторы

Конструктор – это специальная функция, которая строит объект на основе переданных атрибутов. Если вы явно не добавляете никаких конструкторов, то Java все равно создаст

для класса конструктор без аргументов. Если конструктор не инициализирует значения переменных объекта, то они получают значения по умолчанию. Для переменных числовых типов это число 0, а для типа String и классов – это значение `null`.

Если необходимо, чтобы при создании объекта производилась какая-то логика, например, чтобы поля класса получали какие-то определенные значения, то можно определить в классе свои конструкторы. При этом если в классе определяются свои конструкторы, то этот класс лишается конструктора по умолчанию.

```
class Graph {
    private List<Integer>[] adjList;

    public Graph(int V) {
        adjList = new ArrayList<Integer>[V];

        for (int i = 0; i < V; i++) {
            adjList[i] = new ArrayList<Integer>();
        }
    }
}
```

Ключевое слово `this`

Ключевое слово `this` представляет ссылку на текущий экземпляр (объект) класса. Через это ключевое слово мы можем обращаться к переменным и методам объекта, а также вызывать его конструкторы:

```
class Person {
    private String name;

    public Person(String name) {
        this.name = name;
    }

    public Person() {
        this.Person("Nameless");
    }
}
```

Статические члены класса

Кроме обычных полей, методов, свойств классы и структуры могут иметь статические поля и методы. Для обращения к ним необязательно создавать объект класса.

Пример: класс `Math` (`java.lang.Math`):

```
double x = 5.3;
double y = Math.exp(- x*x / 2);
```

Для объявления статических переменных, констант, методов и инициализаторов перед их объявлением указывается ключевое слово `static`. (Статические инициализаторы предназначены для инициализации статических переменных, либо для выполнения таких действий, которые выполняются при создании самого первого объекта)

```
class Person{
    int age;          // обычное поле
    static int retirementAge; // статическое поле

    static { // статический инициализатор
        retirementAge = 65;
    }

    Person(int age){
        this.age = age;
        System.out.println("Constructor");
    }

    static void checkAge(Person p) { // статический метод
        if (p.age >= retirementAge) {
            System.out.println("Уже на пенсии");
        } else {
            System.out.printf("До пенсии осталось %d лет.\n", retirementAge -
p.age);
        }
    }
}
```

Следует учитывать, что статические методы могут обращаться только к статическим членам класса. Обращаться к нестатическим методам и полям этого же класса внутри статического метода мы не можем. Поэтому, например, чтобы вызвать в методе `main` другие методы, которые определены в одном классе с методом `main`, они также должны иметь модификатор `static`.

Модификаторы доступа, инкапсуляция

Инкапсуляция – это объединение данных и поведения в одну единицу кода (пакет/модуль/класс и т.д.) и сокрытие деталей реализации (в нашем случае реализации полей и

методов) от пользователей объекта. Конкретный объект, являющийся экземпляром класса, будет иметь определённые значения своих полей. Набор этих значений представляет собой текущее состояние объекта. При каждом вызове метода объекта его состояние может измениться.

Все компоненты класса в языке Java имеют модификаторы доступа. Модификаторы доступа управляют видимостью классов, методов и переменных, тем самым определяя, как к компонентам класса можно получить доступ из других частей программы. Они управляют тем, откуда можно получить доступ к конкретному классу, методу или переменной.

В Java используются следующие модификаторы доступа:

- **public** : публичный, общедоступный класс или компонент класса. Поля и методы, объявленные с модификатором `public`, видны другим классам из текущего пакета и из внешних пакетов, вообще в любом месте программы. В исходном файле может быть только один класс с модификатором `public`, но количество классов с другими модификаторами (или без них) может быть любым. Если в файле есть класс с модификатором `public`, то имя файла должно совпадать с именем этого класса.
- **private** : закрытый класс или компонент класса, противоположность модификатору `public`. Закрытый класс или компонент класса доступен только из кода в том же классе.
- **protected** : такой класс или компонент класса доступен из любого места в текущем классе или пакете или в производных классах, даже если они находятся в других пакетах
- **Модификатор по умолчанию**. Отсутствие модификатора у поля или метода класса предполагает применение к нему модификатора по умолчанию. Такие поля или методы видны всем классам в текущем пакете.

Модификаторы	Текущий класс	Текущий пакет	Производный класс из другого пакета	Непроизводный класс из другого пакета
private				
Без модификатора				
protected				
public				

Внутренние классы, локальные классы, их особенности и примеры

Внутренние классы – это классы, объявленные внутри другого класса. Они имеют доступ ко всем полям и методам внешнего класса, включая `private`. Не могут содержать статические поля/методы. Чаще всего используются, когда логика тесно связана с внешним классом и не имеет смысла существовать отдельно. Для создания объекта внутреннего класса требуется объект внешнего класса.

Локальные классы объявляются внутри метода, конструктора или блока кода. Они применяются, когда класс нужен только в пределах одного метода. Локальные классы имеют доступ к полям внешнего класса и к локальным переменным метода (если после инициализации их значение не изменяется). Не могут иметь модификаторов доступа.

```
public class Outer {
    private int outerField = 10;

    class Inner {
        void printOuterField() {
            System.out.println("Outer field: " + outerField);
        }
    }

    void methodWithLocalClass() {
        int localVariable = 5;

        class Local {
            void printSum() {
                System.out.println("Sum: " + (outerField + localVariable));
            }
        }

        Local local = new Local();
        local.printSum();
    }

    public static void main(String[] args) {
        Outer outer = new Outer();
        Inner inner = outer.new Inner();
        inner.printOuterField();
        outer.methodWithLocalClass();
    }
}
```

3. Наследование. Преобразования типов между родителем и наследником. Переопределение методов у наследника. Абстрактные классы. Интерфейсы. Принцип подстановки Лисков.

Одним из ключевых аспектов объектно-ориентированного программирования является наследование. С помощью наследования можно расширить функционал уже имеющихся классов за счет добавления нового функционала или изменения старого.

Чтобы унаследовать один класс от другого, надо использовать после имени класса-наследника ключевое слово `extends`, после которого идет имя базового класса:

```
class Base {}

class Derived extends Base {}
```

Мы можем использовать функционал базового класса, обращаясь к объекту производного класса. Также можно в производном классе расширить функционал базового – добавить свои поля и методы.

Если в базовом классе определены конструкторы, то в конструкторе производного класса необходимо вызвать один из конструкторов базового класса с помощью ключевого слова `super`, в который передаются значения для параметров конструкторов.

```
class Base {
    String strField;

    public Base(String str) {
        this.strField = str;
    }
}

class Derived extends Base {
    int intField;

    public Derived(String str, int num) {
        super(str);
        this.intField = num;
    }
}
```

Стоит отметить, что при определении конструктора в производном классе до версии Java 25 вызов конструктора суперкласса должны был выполняться ДО всех остальных

инструкций.

Преобразования типов между родителем и наследником

Преобразование от наследника к родителю происходит автоматически, так как дочерний класс всегда содержит в себе всё то, что есть в родительском.

```
Base base = new Derived("six", 7);
```

После такого преобразования будут доступны только методы, которые определены в базовом классе.

Преобразование от родителя к наследнику не происходит автоматически, так как компилятор не может гарантировать, что в переменной родительского типа реально лежит нужный подкласс. Это придётся делать явно, причём если попытаться привести родителя к тому наследнику, которым он на самом деле не является, возникает `ClassCastException`.

```
Animal animal = new Cat();

Cat cat = (Cat) animal;
cat.meow(); // Теперь есть доступ к методам, определённым в Cat

Dog dog = (Dog) animal; // ClassCastException
```

Для безопасной проверки в Java есть оператор `instanceof`:

```
if (animal instanceof Cat) {
    Cat cat = (Cat) animal;
    cat.meow();
}

// С Java 14 есть PatternMatching и можно делать так:
if (animal instanceof Cat cat) {
    cat.meow();
}
```

Переопределение методов у наследника

Производный класс может переопределять методы, которые унаследованы от базового класса. При переопределении метода производный класс определяет метод с той же сигнатурой (имя, аргументы) и аннотацией `@Override` (аннотация не обязательна):

```

class DayPhaser extends Phaser {
    @Override
    protected boolean onAdvance(int phase, int registeredParties) {
        if (registeredParties == 0) {
            return true;
        }

        System.out.println("\n==== ДЕНЬ " + (phase + 2) + " =====");
        return false;
    }
}

```

При переопределении метода он должен иметь уровень доступа не меньше, чем уровень доступа в базовом классе. Например, если в базовом классе метод имеет модификатор `public`, то и в производном классе метод должен иметь модификатор `public`.

С помощью ключевого слова `super` мы также можем обратиться к реализации базового класса:

```

class DayPhaser extends Phaser {
    @Override
    protected boolean onAdvance(int phase, int registeredParties) {
        System.out.println("\n==== ДЕНЬ " + (phase + 2) + " =====");
        return super.onAdvance(phase, registeredParties);
    }
}

```

Можно запретить переопределение методов или наследоваться от класса с помощью ключевого слова `final`:

```

final class FinalClassExample {}

class Base {
    int intField = 3;

    final void foo() {
        System.out.println(" _ _ ");
    }
}

```

Абстрактные классы

Кроме обычных классов в Java есть абстрактные классы. Абстрактный класс похож на обычный класс. В абстрактном классе также можно определить поля и методы, но в то же время нельзя создать объект или экземпляр абстрактного класса. Абстрактные классы призваны предоставлять базовый функционал для классов-наследников. А производные классы уже реализуют этот функционал. При определении абстрактных классов используется ключевое слово `abstract`:

```
abstract class имяКласса {  
    // Что-то написать сюда  
}
```

Наряду с абстрактными классами в языке Java есть абстрактные методы. Такие методы также предваряются модификатором `abstract`, но при этом не имеют тела – не выполняют никаких конкретных действий:

```
abstract void foo(int a);
```

Производный класс обязан переопределить и реализовать все абстрактные методы, которые имеются в базовом абстрактном классе. Также следует учитывать, что если класс имеет хотя бы один абстрактный метод, то данный класс должен быть определен как абстрактный (и производные абстрактные классы не обязаны реализовывать все абстрактные методы).

Чтобы не писать ещё банальный пример, вот мой абстрактный класс альпиниста из задачи про шерпов. Базовые правила симуляции и порядок действий определяются моделью задачи, однако стратегию для перекладывания еды нужно в каждом наследнике написать свою (это можно было сделать лучше, но даже в таком виде, это показательный пример):

```
abstract class HeightBoundClimber extends Thread {  
    public static final int DAYS_TO_SUMMIT = 8;  
    public static final int BACKPACK_SIZE = 4;  
  
    public static final AtomicBoolean someoneDied = new AtomicBoolean(false);  
  
    protected static DayPhaser phaser;  
    protected static SharedEnvironment env;  
  
    protected int height = 0;  
    protected int backpack = BACKPACK_SIZE;  
    protected boolean ascending = true;
```

```

protected final int maxHeight;

protected HeightBoundClimber(String name, int maxHeight) {
    super(name);
    this.maxHeight = maxHeight;
    phaser.register();
}

@Override
public final void run() {
    try {
        while (true) {
            pickFood();

            if (!eat()) break;

            move();

            dropFood();

            phaser.arriveAndAwaitAdvance();

            if (isFinished()) break;
        }
    } finally {
        phaser.arriveAndDeregister();
    }
}

protected abstract void pickFood();
protected abstract void dropFood();

protected boolean eat() {
    if (backpack == 0) {
        someoneDied.set(true);
        System.out.println(getName() + " погиб в горах (нет еды)");
        return false;
    }
    backpack--;
    return true;
}

protected void move() {
    if (ascending) height++;
    else height--;
}

```

```

        protected boolean isFinished() {
            if (!ascending && height == 0) {
                System.out.println(getName() + " вернулся домой.");
                return true;
            }
            return false;
        }
    }
}

```

Интерфейсы

Механизм наследования очень удобен, но он имеет свои ограничения. В частности, в Java нет множественного наследования (от классов).

Подобную проблему частично позволяют решить интерфейсы. Интерфейсы определяют "контракт" т.е. множество методов (по умолчанию, `public abstract`), не имеющих конкретной реализации, которые затем реализуют классы, применяющие эти интерфейсы. И один класс может применить множество интерфейсов.

Чтобы определить интерфейс, используется ключевое слово `interface`.

```

interface Animal {
    public void makeSound();
}

```

Чтобы указать, что класс реализует интерфейс, после названия класса указывается ключевое слово `implements`, после которого идет название интерфейса:

```

class Cat implements Animal {
    @Override
    public void makeSound() {
        System.out.println("Meow");
    }
}

```

Интерфейсы в языке Java также могут иметь методы по умолчанию (default methods) – методы с реализацией по умолчанию, которая используется, если класс, реализующий данный интерфейс, не реализует метод. Метод по умолчанию – это обычный метод без модификаторов (формально имеет модификатор `public`), который помечается ключевым словом `default`:

```
interface Animal {  
    public void makeSound();  
    default void info() {  
        System.out.println("Is animal.");  
    }  
}
```

Принцип подстановки Лисков

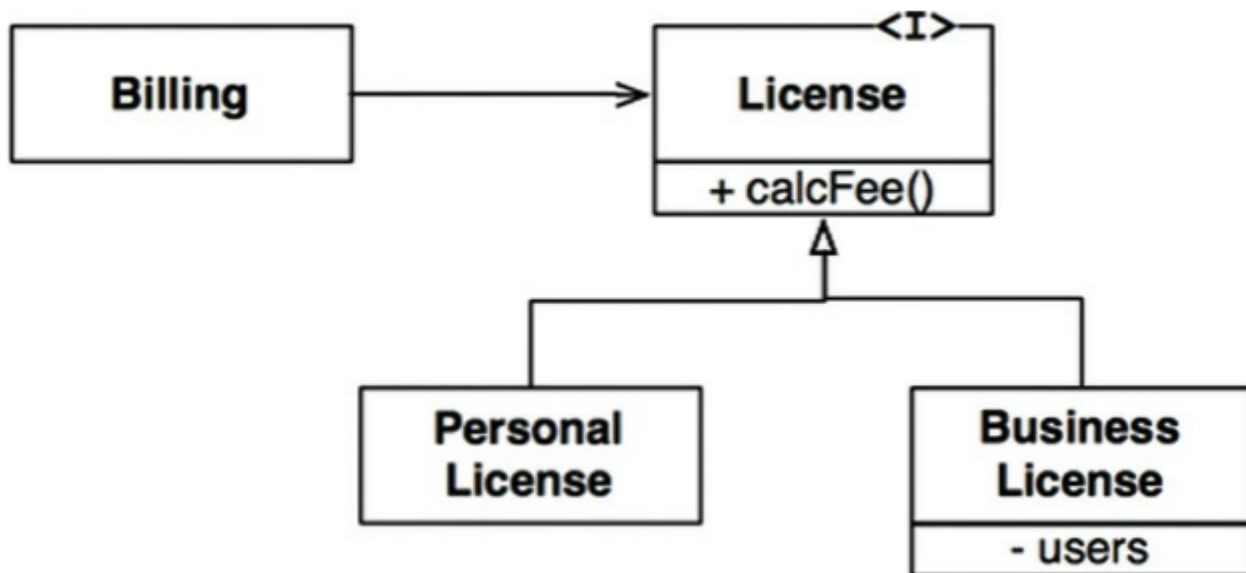
Тут будет частично цитироваться книга "Чистая Архитектура" Роберта Мартина. Рекомендую прочитать 9-ую главу из неё.

В 1988 году Барбара Лисков написала следующие строки с формулировкой определения подтипов:

Если для каждого объекта $o1$ типа S существует какой-то объект $o2$ типа T , что для всех программ P , определённых в типах T , поведение P не изменяется при подстановке $o1$ вместо $o2$, то S является подтипом T .

Проще говоря: если код зависим от интерфейса, то подстановка любой его реализации не должна ломать код.

Пример архитектуры, соблюдающей этот принцип:



Есть интерфейс License. В нём определён метод calcFee(), который вызывается приложением Billing. Существует два "подтипа" класса License: PersonalLicense и BusinessLicense. Они реализуют разные алгоритмы расчёта лицензионных отчислений. Оба подтипа могут служить заменой для типа License.

4. Исключения. Иерархия исключений. Проверяемые и непроверяемые исключения. Конструкция try with resources.

Нередко в процессе выполнения программы могут возникать ошибки, при том необязательно по вине разработчика. Некоторые из них трудно предусмотреть или предвидеть, а иногда и вовсе невозможно. Подобные ситуации называются исключениями.

В языке Java предусмотрены специальные средства для обработки подобных ситуаций. Одним из таких средств является конструкция **try...catch...finally***. При возникновении исключения в блоке `try` управление переходит в блок `catch`, который может обработать данное исключение (можно добавлять несколько блоков `catch` под разные виды исключений). После выполнения всех блоков `try/catch` выполняется блок `finally`. Если подходящего блока `catch` не найдено, то пользователю отображается сообщение о необработанном исключении, а дальнейшее выполнение программы останавливается.

```
try {  
    /// smth  
} catch (Exception e) {  
    /// smth  
} finally {  
    /// smth  
}
```

Единственным обязательным блоком в этой конструкции является `try`. Компилятор позволит написать либо `catch`, либо `finally`, а не оба. Собственно, в связи с этим чаще всего встречается именно **try...catch** конструкция.

Чтобы бросить исключение в своём коде применяется оператор `throw`.

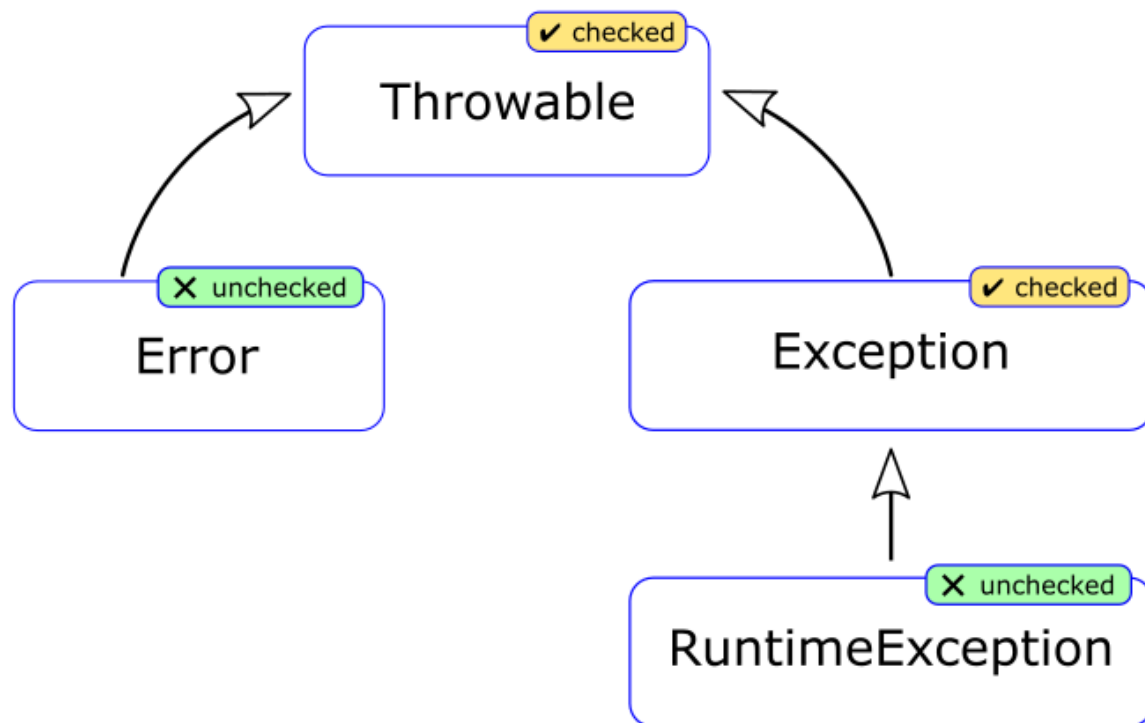
```
public class Main {  
    public static void main(String args[]) {  
        Integer a = 555;  
        Integer b = 555;  
  
        if (a != b) {  
            throw new RuntimeException("Це ловушка в жабе.");  
        }  
    }  
}
```

Да это реально упадёт. Прикол в том, что Integer – обёртка над int и на самом деле является ссылкой на объект в памяти. Объекты для чисел от -128 до 127 создаются при запуске программы (IntegerCache). Ссылки на объект типа Integer со значением в этом диапазоне будут на один и тот же объект в памяти. Поэтому поменяв значение, например, на 55 код не упадёт.

Если метод генерирует проверяемое исключение, то нам надо либо обработать это исключение, либо сообщить компилятору, что метод генерирует данное исключение. В случае, если метод не обрабатывает его сам, в объявлении метода необходимо указать тип исключения, который генерирует метод. Для этого используется оператор throws в следующем виде:

```
public void funnyIslandMethod() throws NumberFormatException, FileNotFoundException
```

Иерархия исключений. Проверяемые и непроверяемые исключения.



Все исключения в Java являются объектами. Базовым классом для них является Throwable (наследник Object). Любой класс, который наследуется от Throwable, считается исключением. Блоки try ... catch и оператор throw будут работать с Throwable, но это не рекомендуется наследоваться от него напрямую. Иерархия чётко разделяет все

исключительные ситуации на две ветви – те, которые программа должна обрабатывать, и те, которые не должна, а если точнее:

- **Error** – ошибка, которая означает серьезную проблему в программе, и при её возникновении восстанавливать работу программы не предполагается. Наследуется от **Throwable**. Примеры: переполнения стека вызовов (**StackOverflowError**), недостаток памяти (**OutOfMemoryError**)
- **Exception** – проверяемое исключение, наследуется от **Throwable**. Пример: поток прерван другим потоком (**InterruptedException**)
- **RuntimeException** – непроверяемое исключение, наследуется от **Exception**. Примеры: деление на ноль (**ArithmeticException**), использование пустой ссылки (**NullPointerException**), индекс вне границ массива (**IndexOutOfBoundsException**), - использование неверного аргумента при вызове метода (**IllegalArgumentException**).

Наследники **Error** и **RuntimeException** называют непроверяемыми. Компилятор не проверяет факт их обработки в **try...catch** блоках.

Наследники **Exception** – проверяемые исключения, компилятор не позволит запустить программу, если она не обрабатывает их.

Конструкция **try with resources**

При завершении работы с потоком (I/O Stream, а не Thread) его надо закрыть с помощью метода **close()** , который определен в интерфейсе **Closeable** : **void close() throws IOException** . Этот интерфейс уже реализуется в классах **InputStream** и **OutputStream** , а через них и во всех классах потоков. При закрытии потока освобождаются все выделенные для него ресурсы, например, файл. В случае, если поток окажется не закрыт, может происходить утечка памяти.

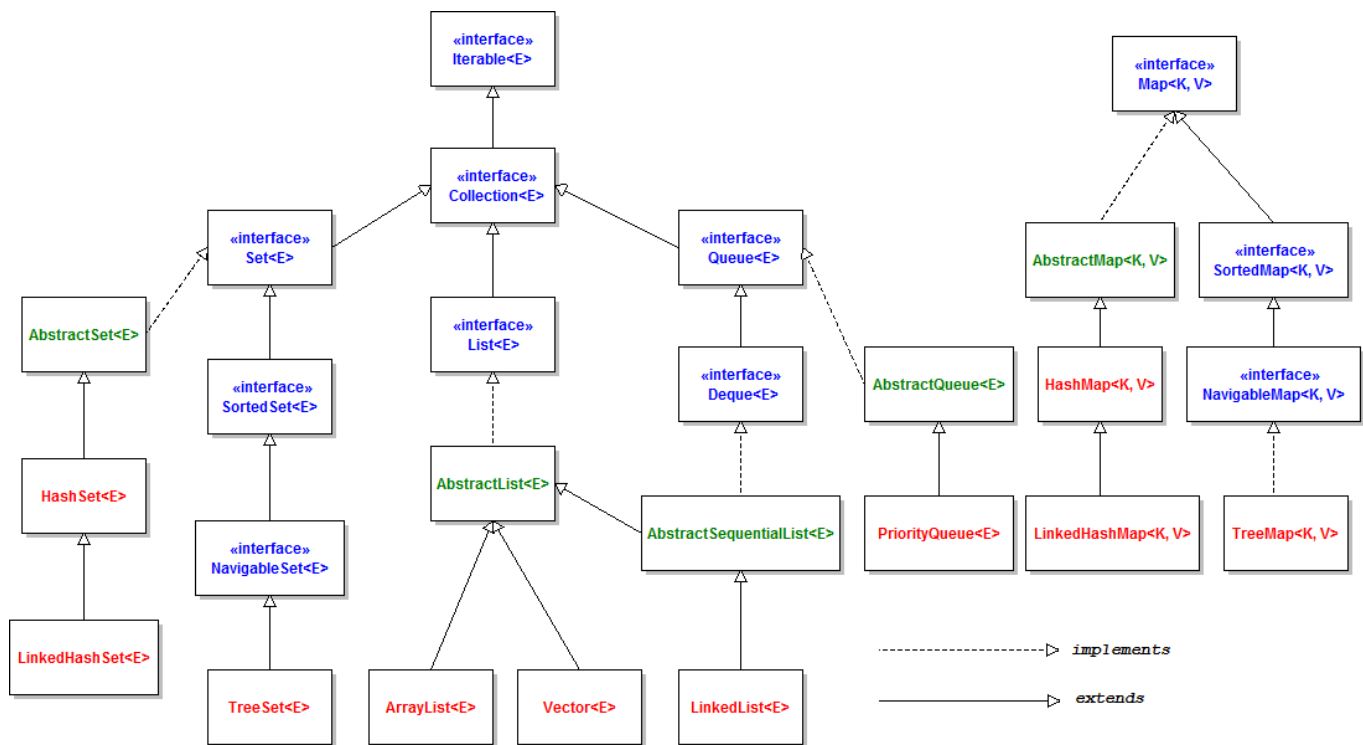
Если не использовать конструкцию **try-with-resources** , придётся закрывать вручную, и выйдет много кода. Для ресурсов, реализующих интерфейс **Closeable** (или **AutoClosable**) закрытие вызывается автоматически, сокращая код до:

```
try (FileInputStream fin = new FileInputStream("text.txt")) {  
    /// read and do smth  
} catch (IOException e) {  
    System.out.println(e.getMessage())  
}
```

В этой конструкции также можно открыть сразу несколько ресурсов, через точку с запятой:

```
try (FileInputStream fin = new FileInputStream("text.txt");
    FileOutputStream fout = new FileOutputStream("out.txt")) {
    /// read and do smth
} catch (IOException e) {
    System.out.println(e.getMessage())
}
```

5. Стандартные коллекции: ArrayList, LinkedList, HashMap, TreeMap, LinkedHashMap. Их основные методы и внутреннее устройство.



Вот +- полная иерархия коллекций в Java. Зелёные классы – абстрактные, синие – интерфейсы, красные – имплементации.

Collection

Начнём со схемы слева, она начинается с интерфейса `Iterable<E>`. В нём определён метод `iterator()`, возвращающий итератор `Iterator<E>`. т.е. если какой-то класс/интерфейс наследует `Iterable<E>`, то по нему можно пройтись в цикле `for`: `for (E element : iterable)`.

Интерфейс `Collection` расширяет `Iterable`. В отличие от `Iterable`, он олицетворяет собой какую-то коллекцию с конечным количеством элементов. Из методов, объявленных в

`Collection<E>` СТОИТ ВЫДЕЛИТЬ:

- `int size()`
- `boolean isEmpty()`
- `boolean contains(Object o)`
- `boolean add(E e)`
- `boolean remove(E e)`
- `void clear()`

Правее на схеме можно заметить интерфейс `Queue` для FIFO структур данных. Из него упомяну методы: `E peek()` и `E poll()`. В принципе про эту ветвь иерархии говорить имеет смысл, только чтобы упомянуть, что двусвязный список `LinkedList<E>` реализует как интерфейс `List<E>`, так и `Deque<E>`. На этом всё с доп. инфой, перейдём к основному содержанию билета.

Списки: `ArrayList` и `LinkedList`

Для создания простых списков применяется интерфейс `List`, который расширяет функциональность интерфейса `Collection`. Некоторые наиболее часто используемые методы интерфейса `List`:

- `void add(int index, E obj)` : добавляет в список по индексу `index` объект `obj`
- `void addFirst(E obj)` : добавляет объект в начало списка
- `void addLast(E obj)` : добавляет объект в конец списка
- `boolean addAll(int index, Collection<? extends E> col)` : добавляет в список по индексу `index` все элементы коллекции `col`. Если в результате добавления список был изменен, то возвращается `true`, иначе возвращается `false`
- `E get(int index)` : возвращает объект из списка по индексу `index`
- `E getFirst()` : возвращает первый элемент списка
- `E getLast()` : возвращает последний элемент списка
- `int indexOf(Object obj)` : возвращает индекс первого вхождения объекта `obj` в список. Если объект не найден, то возвращается `-1`
- `int lastIndexOf(Object obj)` : возвращает индекс последнего вхождения объекта `obj` в список. Если объект не найден, то возвращается `-1`
- `ListIterator<E> listIterator()` : возвращает объект `ListIterator` для обхода элементов списка
- `static <E> List<E> of(элементы)` : создает из набора элементов объект `List`
- `E remove(int index)` : удаляет объект из списка по индексу `index`, возвращая при этом удаленный объект
- `E removeFirst()` : удаляет первый элемент из списка

- `E removeLast()` : удаляет последний элемент из списка
- `E set(int index, E obj)` : присваивает значение объекта `obj` элементу, который находится по индексу `index`
- `void sort(Comparator<? super E> comp)` : сортирует список с помощью компаратора `comp`
- `List<E> subList(int start, int end)` : получает набор элементов, которые находятся в списке между индексами `start` и `end`
- `static <E> List<E>of(E... elements)` : возвращает неизменяемый список (тип `java.util.ImmutableCollections$ListN`), который содержит элементы `elements`.

Класс `ArrayList` представляет обобщенную коллекцию, которая наследует свою функциональность от класса `AbstractList` и применяет интерфейс `List`. Это, по сути, динамический массив.

Размер по умолчанию – 10, но его также можно задать в конструкторе. При расширении, новый размер вычисляется по формуле: $capacity = capacity * 3 / 2$, то есть увеличивается на 50%.

В Java нет доступа к `capacity`, поэтому, чтобы продемонстрировать это в эксперименте, пришлось взять небольшой трюк с reflection со [StackOverflow](#). И то это работает только на старых версиях (я запускал на Java 12):

```
import java.util.ArrayList;
import java.util.List;
import java.lang.reflect.Field;

public class Main
{
    public static void main(String[] args) throws Exception {
        List<Integer> list = new ArrayList<>();

        for (int i = 0; i < 1024; i++) {
            list.add(i);
            System.out.println(i + " " + getCapacity(list));
        }
    }

    static int getCapacity(List al) throws Exception {
        Field field = ArrayList.class.getDeclaredField("elementData");
        field.setAccessible(true);
        return ((Object[]) field.get(al)).length;
    }
}
```

```
0 10
...
9 10
10 15
...
14 15
15 22
...
21 22
22 33
...
32 33
33 49
...
49 73
...
73 109
...
109 163
...
1023 1234
```

Обобщенный класс `LinkedList<E>` представляет структуру данных в виде двусвязного списка. Он наследуется от класса `AbstractSequentialList` и реализует интерфейсы `List`, `Deque` и `Queue`. То есть он соединяет функциональность работы со списком и функциональность очереди.

Класс `LinkedList` имеет следующие конструкторы:

- `LinkedList()` : создает пустой список
- `LinkedList(Collection<? extends E> col)` : создает список, в который добавляет все элементы коллекции `col`

Из методов ранее упомянутых интерфейсов, реализуемых двусвязным списком, следует выделить:

- `addFirst()` / `offerFirst()`: добавляет элемент в начало списка
- `addLast()` / `offerLast()`: добавляет элемент в конец списка
- `removeFirst()` / `pollFirst()`: удаляет первый элемент из начала списка
- `removeLast()` / `pollLast()`: удаляет последний элемент из конца списка
- `getFirst()` / `peekFirst()`: получает первый элемент
- `getLast()` / `peekLast()`: получает последний элемент

Иерархия Map

Словари (также называемые картами и отображениями) позволяют сопоставить с определенным ключом некоторое значение. В языке Java для определения базового функционала словарей имеется интерфейс `Map<K, V>`. Он представляет структуру данных, где каждый элемент представляет пару "ключ-значение". При этом в рамках объекта `Map` все ключи уникальные.

Непосредственно итератора у них нет (не реализуют `Iterable`), но того же эффекта можно достичь другими способами:

```
for (Map.Entry<K, V> entry : map.entrySet()) {  
    K key = entry.getKey();  
    V value = entry.getValue();  
}
```

Среди методов интерфейса `Map` можно выделить следующие:

- `int size()` . Возвращает количество записанных пар ключ-значение.
- `boolean isEmpty()` . Возвращает `true`, если `Map` пустая.
- `boolean containsKey(Object key)` . Возвращает `true`, если в `Map` присутствует указанный ключ.
- `boolean containsValue(Object value)` . Возвращает `true`, если в `Map` присутствует указанное значение.
- `V get(Object key)` . Возвращает значение по указанному ключу.
- `V put(K key, V value)` . Записывает пару ключ-значение. Возвращает предыдущее значение, которое было под этим ключом, или `null`, если ключ до этого отсутствовал.
- `V putIfAbsent(K k, V v)` : помещает в коллекцию новый объект с ключом `k` и значением `v`, если в коллекции еще нет элемента с подобным ключом.
- `V remove(Object key)` . Удаляет пару ключ-значение по переданному ключу. Возвращает значение, которое было удалено, или `null`, если по ключу ничего не было записано.
- `void clear()` . Удаляет все пары ключ-значение.
- `Set<K> keySet()` . Возвращает `Set` из всех ключей, которые записаны в `Map`.
- `Collection<V> values()` . Возвращает коллекцию из всех значений, которые записаны в `Map`.
- `Set<Map.Entry<K, V>> entrySet()` . Возвращает `Set` из всех пар ключ-значение.
- `void clear()` : очищает коллекцию
- `boolean replace(K key, V oldValue, V newValue)` : заменяет значение для указанного ключа `key` на `newValue`, если только старое значение равно `oldValue`

HashMap и LinkedHashMap

HashMap – реализация Map как хеш-таблицы. Для корректной работы ключи должны правильно реализовывать equals/hashCode. При этом значениям реализовывать эти методы необязательно.

Под капотом это хеш-таблица на цепочках. Но стоит выделить пару особенностей:

- Изначально массив длины `capacity`, содержащий связные списки – бакеты.
- Если в бакете оказалось больше восьми ключей, то он преобразуется из связного списка в красно-чёрное дерево.
- Ресайз происходит, когда количество элементов превышает $(loadFactor * capacity)$. Тогда `capacity` увеличивается в двое. Значения по умолчанию: `loadFactor=0.75`, `capacity=16`

У HashMap еще имеется наследник LinkedHashMap. Работает точно так же, только элементы гарантировано хранятся в том порядке, в каком их добавили.

Узел `LinkedHashMap.Entry` наследует `HashMap.Node` и добавляет поля: `Entry<K,V> before` и `Entry<K,V> after` – ссылки на предыдущий и следующий вставленные элементы.

TreeMap

Осталось рассмотреть самую правую ветвь иерархии с картинки.

Интерфейс `SortedMap` расширяет `Map` и позволяет обеспечить полное упорядочение своих ключей. По умолчанию элементы упорядочиваются в соответствии с естественным порядком ключей (по возрастанию) или с помощью компаратора.

`SortedMap` добавляет ряд методов:

- `K firstKey()` : возвращает ключ первого элемента коллекции
- `K lastKey()` : возвращает ключ последнего элемента коллекции
- `SortedMap<K, V> headMap(K end)` : возвращает объект `SortedMap`, который содержит все элементы оригинального `SortedMap` вплоть до элемента с ключом `end`
- `SortedMap<K, V> tailMap(K start)` : возвращает объект `SortedMap`, который содержит все элементы оригинального `SortedMap`, начиная с элемента с ключом `start`
- `SortedMap<K, V> subMap(K start, K end)` : возвращает объект `SortedMap`, которые содержит все элементы оригинального `SortedMap` вплоть от элемента с ключом `start` до элемента с ключом `end`

Интерфейс `NavigableMap` расширяет интерфейс `SortedMap` и обеспечивает возможность получения элементов коллекции относительно других элементов (условно говоря, навигацию). Его основные методы:

- `Map.Entry<K, V> ceilingEntry(K key)` : возвращает элемент с наименьшим ключом `k`, который больше или равен ключу `key` ($k \geq \text{key}$). Если такого ключа нет, то возвращается `null`.
- `Map.Entry<K, V> floorEntry(K key)` : возвращает элемент с наибольшим ключом `k`, который меньше или равен ключу `key` ($k \leq \text{key}$). Если такого ключа нет, то возвращается `null`.
- `Map.Entry<K, V> higherEntry(K key)` : возвращает элемент с наименьшим ключом `k`, который больше ключа `key` ($k > \text{key}$). Если такого ключа нет, то возвращается `null`.
- `Map.Entry<K, V> lowerEntry(K key)` : возвращает элемент с наибольшим ключом `k`, который меньше ключа `key` ($k < \text{key}$). Если такого ключа нет, то возвращается `null`.
- `Map.Entry<K, V> firstEntry()` : возвращает первый элемент коллекции
- `Map.Entry<K, V> lastEntry()` : возвращает последний элемент коллекции
- `Map.Entry<K, V> pollFirstEntry()` : возвращает и одновременно удаляет первый элемент из коллекции
- `Map.Entry<K, V> pollLastEntry()` : возвращает и одновременно удаляет последний элемент из коллекции
- `K ceilingKey(K key)` : возвращает наименьший ключ `k`, который больше или равен ключу `key` ($k \geq \text{key}$). Если такого ключа нет, то возвращается `null`.
- `K floorKey(K key)` : возвращает наибольший ключ `k`, который меньше или равен ключу `key` ($k \leq \text{key}$). Если такого ключа нет, то возвращается `null`.
- `K lowerKey(K key)` : возвращает наибольший ключ `k`, который меньше ключа `key` ($k < \text{key}$). Если такого ключа нет, то возвращается `null`.
- `K higherKey(K key)` : возвращает наименьший ключ `k`, который больше ключа `key` ($k > \text{key}$). Если такого ключа нет, то возвращается `null`.
- `NavigableSet<K> descendingKeySet()` : возвращает объект `NavigableSet`, который содержит все ключи коллекции в обратном порядке
- `NavigableMap<K, V> descendingMap()` : возвращает коллекцию `NavigableMap`, которое содержит все элементы в обратном порядке
- `NavigableSet<K> navigableKeySet()` : возвращает объект `NavigableSet`, который содержит все ключи коллекции
- `NavigableMap<K, V> headMap(K upperBound, boolean incl)` : возвращает коллекцию `NavigableMap`, которое содержит все элементы оригинальной коллекции `NavigableMap` вплоть от элемента с ключом `upperBound`. Параметр `incl` при значении `true` указывает, что элемент с ключом `upperBound` также включается в выходной набор.
- `NavigableMap<K, V> tailMap(K lowerBound, boolean incl)` : возвращает коллекцию `NavigableMap`, которое содержит все элементы оригинальной коллекции `NavigableMap`, начиная с элемента с ключом `lowerBound`. Параметр `incl` при

значении true указывает, что элемент с ключом lowerBound также включается в выходной набор.

- `NavigableMap<K, V> subMap(K lowerBound, boolean lowIncl, K upperBound, boolean highIncl)` : возвращает коллекцию `NavigableMap`, которое содержит все элементы оригинальной коллекции `NavigableMap` от элемента с ключом `lowerBound` до элемента с ключом `upperBound`. Параметры `lowIncl` и `highIncl` при значении true включают в выходной набор элементы с ключами `lowerBound` и `upperBound` соответственно.

Встроенной реализацией выше упомянутых интерфейсов является класс `TreeMap<K, V>`. Под капотом там красно-чёрное дерево.

Отметим, что методы вроде `keySet()`, `values()` и `entrySet()` в `TreeMap` не создают новые коллекции. Они возвращают представления. Все операции над ними напрямую транслируются в запросы к дереву:

```
import java.util.NavigableMap;
import java.util.TreeMap;

public class MyClass {
    public static void main(String args[]) {
        NavigableMap<Integer, Integer> map = new TreeMap<>();
        map.put(2, 0);
        map.put(3, 0);
        map.put(4, 0);

        var tmap = map.tailMap(3, true);
        tmap.remove(3);
        tmap.replace(4, 0, 1);

        System.out.println(map.get(2)); // 0
        System.out.println(map.get(3)); // null
        System.out.println(map.get(4)); // 1
    }
}
```

Класс `TreeMap` имеет следующие конструкторы:

- `TreeMap()` : создает пустое коллекцию в виде дерева
- `TreeMap(Map<? extends K, ? extends V> map)` : создает дерево, в которое добавляет все элементы из коллекции `map`
- `TreeMap(SortedMap<K, ? extends V> smap)` : создает дерево, в которое добавляет все элементы из коллекции `smap`
- `TreeMap(Comparator<? super K> comparator)` : создает пустое дерево, где все добавляемые элементы впоследствии будут отсортированы компаратором.

6. Лямбда-функции. Элементы функционального программирования. Захват окружения. Ссылки на методы.

Лямбда-функции и функциональные интерфейсы

Lambda-выражения – это анонимные функции, т.е. метод без объявления. Лямбды в Java обычно имеют следующий синтаксис (аргументы) -> (тело) . Например:

```
(int a, int b) -> { return a + b; }

() -> System.out.println("Hello World");

(String s) -> {
    /// some other code idk
    System.out.println(s);
};

() -> 42

() -> { return 3.1415 };

```

- Lambda-выражения могут иметь от 0 и более входных параметров.
- Тип параметров можно указывать явно либо может быть получен из контекста.
- Когда параметр один, если тип не указывается явно, скобки можно опустить.
Например, `x -> return x * x`
- Если тело состоит из одного оператора, его можно не заключать в фигурные скобки, а возвращаемое значение можно указывать без ключевого слова `return` . В противном случае фигурные скобки обязательны (блок кода), а в конце надо указывать возвращаемое значение с использованием ключевого слова `return` (в противном случае типом возвращаемого значения будет `void`).

Функциональный интерфейс (functional interface) – это интерфейс, который содержит ровно один абстрактный метод. Каждое lambda-выражение может быть неявно привязано к какому-нибудь функциональному интерфейсу.

`java.lang Runnable` – это пример функционального интерфейса. В нем объявлен только один метод `void run()` . К нему можно привязать любую лямбду, не принимающую аргументов и возвращающую `void` :

```
Runnable r1 = () -> System.out.println("Hello");

```

```
int[] array = new int[3];
Runnable r2 = () -> {
    array[0]++;
    array[1]--;
};

r1.run();
r2.run();
```

В `java.util.function` есть функциональные интерфейсы, покрывающие большинство необходимых случаев использования:

Интерфейс	Метод	Описание
<code>Predicate<T></code>	<code>boolean test(T t)</code>	проверяет соблюдение некоторого условия
<code>Consumer<T></code>	<code>void accept(T t)</code>	выполняет некоторое действие над объектом типа T, при этом ничего не возвращая
<code>Function<T, R></code>	<code>R apply(T t)</code>	функция перехода от объекта типа T к объекту типа R
<code>BiFunction<T, U, R></code>	<code>R apply(T t, U u)</code>	представляет функцию с двумя параметрами типа T и U и результатом типа R
<code>Supplier<T></code>	<code>T get()</code>	не принимает никаких аргументов, но должен возвращать объект типа T
<code>UnaryOperator<T></code>	<code>T apply(T t)</code>	принимает в качестве параметра объект типа T, выполняет над ними операции и возвращает результат операций в виде объекта типа T
<code>BinaryOperator<T></code>	<code>T apply(T t1, T t2)</code>	принимает в качестве параметра два объекта типа T, выполняет над ними бинарную операцию и возвращает ее результат также в виде объекта типа T

Ещё простой пример использования:

```
import java.util.Random;
import java.util.function.Supplier;

enum Interval {
    MORNING(0),
    AFTERNOON(1),
    EVENING(2),
    NIGHT(3);
}
```

```

    final int priority;

    Interval(int priority) {
        this.priority = priority;
    }
}

public class Main {
    public static void main(String[] args) {
        Random random = new Random(42);

        Supplier<Interval> nextInterval = () -> {
            return Interval.values()[random.nextInt(0, 4)];
        };

        System.out.println(nextInterval.get());
        System.out.println(nextInterval.get());
        System.out.println(nextInterval.get());
    }
}

```

Для определения своего функционального интерфейса нужно объявить интерфейс с одним методом. Ещё можно добавить аннотацию `@FunctionalInterface`:

```

@FunctionalInterface
interface Unpack<T> {
    T get(Optional<T> opt);
}

```

При необходимости использовать обычный метод как лямбда-выражение применяется оператор двойное двоеточие `::`, это ссылка на метод.

```

Function<String, Integer> len = String::length;
System.out.println(len.apply("Adios"));

```

```

List<Integer> list = Arrays.asList(1, 2, 3, 4, 5, 6, 7);
list.forEach(System.out::println);

```

Помимо статических/нестатических методов классов можно применить `::` так:

- `someObject::someMethod` — ссылка на метод конкретного объекта.
- `ArrayList::new` — ссылка на конструктор.

Касаемо функционального программирования отмечу, что лямбды могут быть аргументами или результатом методов, например, `Collections.sort(List<T> list, Comparator<? super T> c)`.

Захват окружения

В примерах ранее можно заметить, что лямбда может использовать переменные, которые объявлены во вне в более общей области видимости – на уровне класса или метода, в котором лямбда-выражение определено. Однако в зависимости от того, как и где определены переменные, могут различаться способы их использования в лямбдах.

Переменные, объявленные на уровне класса можно получить и изменять:

```
public class Main {
    static int x = 2;

    public static void main(String[] args) {
        Runnable r = () -> {
            x *= x;
            System.out.println(x);
        };

        r.run(); // 4
    }
}
```

Локальные переменные уровня метода мы также можем использовать в лямбдах, но изменять их значение нельзя т.е. переменная должна быть `final` или `effectively final`. Если мы попробуем это сделать, то код не скомпилируется.

```
public class Main {
    public static void main(String[] args) {
        int x = 2;

        Runnable r = () -> {
            // error: local variables referenced from a lambda expression must be
            final or effectively final
            x *= x;
            System.out.println(x);
        };

        r.run();
    }
}
```

7. Optional. Stream API. Промежуточные и терминальные операции. Коллекторы, комбинации коллекторов.

Optional

Класс `Optional<T>` (`java.util.Optional`) – это контейнер, который может содержать, а может не содержать значение определенного типа.

```
public class CustomerRepository {  
    /* поля и конструктор, методы ... */  
  
    public Optional<Customer> findById(CustomerId id) {  
        if (!existsById(id)) {  
            return Optional.empty();  
        }  
        return Optional.ofNullable(customer);  
    }  
}
```

Способы создать `Optional`:

- `Optional.empty()` – Создает пустой контейнер. Внутри него нет ссылки на объект.
- `Optional.of(obj)` – Создает `Optional` с гарантированным значением. Если передать `null`, будет выброшено `NullPointerException`.
- `Optional.ofNullable(obj)` – "Безопасный" способ: если объект есть – создаст заполненный `Optional`, если передана `null`-ссылка – вернет пустой.

Основные методы:

- `boolean isPresent()` и `boolean isEmpty()`
- `T get()` – если не пустой, возвращает объект, иначе `NoSuchElementException`
- `void ifPresent(Consumer<? super T> consumer)` – вызывает переданную лямбду, если значение есть, иначе – ничего.
- `Optional<T> filter(Predicate<? super T> predicate)` – если есть значение и для него выполняется предикат (лямбда вернула `true`), то возвращает непустой контейнер со значением, иначе пустой.
- `Optional<U> map(Function<? super T, Optional<U>> mapper)` – если есть значение, то применяет к нему преобразование `mapper.apply()` и возвращает `Optional<U>` с этим значением, иначе пустой опционал.
- `T orElse(T other)` – если есть значение, то возвращает его, иначе – `other`.

- `T orElseGet(Supplier<? extends T> other)` – если есть значение, то возвращает его, иначе возвращает результат вызова `other.get()` .
- `T orElseThrow(Supplier<? extends X> exceptionSupplier) throws X` – если есть значение, то возвращает его, иначе вызывает `exceptionSupplier` для выброса исключения. В официальной документации предлагается использовать для этого ссылку на конструктор по умолчанию, например:
`opt.orElseThrow(NoSuchElementException::new)` . Если не передать аргумент, то вызовется конструктор `NullPointerException` .

StreamAPI

Начиная с JDK 8 в Java появился новый API – Stream API. Его задача – упростить работу с наборами данных, в частности, упростить операции фильтрации, сортировки и другие манипуляции с данными. Вся основная функциональность данного API сосредоточена в пакете `java.util.stream`. Ключевым понятием в Stream API является поток данных – канал передачи данных из источника данных. В качестве источника могут выступать как файлы, так и коллекции. Одной из отличительных черт Stream API является применение лямбда-выражений, которые позволяют значительно сократить запись выполняемых действий.

Использование потока можно поделить на три части:

1. Его создание
2. Промежуточные операции: сортировка, фильтрация, преобразование и т.п.
3. Терминальную операцию: они приводят поток к финальному виду и закрывают его (в частности, коллекторы).

Создание потока

Есть множество вариантов того, как можно создать поток, например:

Из существующей коллекции (метод `stream()`):

```
List<String> strings = List.of("a", "b", "c");
Stream<String> stream = strings.stream();
```

Используя генератор (передать лямбду вида `Supplier<T>`):

```
Random random = new Random(42);
Stream.generate(random.nextGaussian()).limit(10);
```

Кстати, в `Random` есть готовые методы для создания потоков (см. `doubles()` , `ints()` и `longs()` в [документации](#)).

Range примитивов

Можно задать начальный и конечный элемент с помощью методов `range(firstElement, lastElement)` и `rangeClosed(firstElement, lastElement)`, первый исключает `lastElement` из стрима, второй - включает.

```
IntStream.range(0, 3).boxed(); // 0, 1, 2
LongStream.rangeClosed(0L, 3L).boxed(); // 0L, 1L, 2L, 3L
```

Здесь `boxed()` применяется для получения `Stream<Integer>` из `IntStream`.

Билдер

Со стримами можно использовать паттерн builder и наполнять стрим постепенно:

```
Stream<Integer> builtStream = Stream.<Integer>builder()
    .add(1)
    .add(2)
    .add(3)
    .build();

Stream<Object> builtStream = Stream.builder()
    .add(1)
    .add("hi")
    .add(List.of(1, 2, 3))
    .build();
```

Здесь стоит обратить внимание на то, что в первой строчке мы указываем тип данных, который мы будем класть в стрим, иначе автоматически подставится тип `Object`, и проверка типов элементов не будет осуществляться, как во втором случае.

Также нужно не забыть "собрать" наш стрим методом `build()`, в противном случае будет тип не `Stream<T>`, а `Stream.Builder<T>`.

Промежуточные операции

Промежуточные операции обрабатывают поступающие элементы и возвращают стрим. Промежуточных операторов в цепочке обработки элементов может быть много. Их также называют "ленивыми" (lazy), потому что они не будут выполняться, пока не будет вызван терминальный метод.

Рассмотрим несколько примеров:

- `Stream<T> filter(Predicate<? super T> predicate)` — принимает в качестве параметра лямбду вида `Predicate<T>` и возвращает новый поток из элементов, которые удовлетворяют этому условию.
- `Stream<T> sorted(Comparator<? super T> comparator)` — возвращает отсортированный в соответствии с условием поток.
- `<R> Stream<R> map(Function<? super T, ? extends R> mapper)` — преобразует элементы типа `T` в элементы типа `R` и возвращает поток с элементами `R`.
- `<R> Stream<R> flatMap(Function<? super T, ? extends Stream<? extends R>> mapper)` — позволяет преобразовать элемент типа `T` в несколько элементов типа `R` и возвращает поток с элементами `R`.
- `Stream<T> skip(long n)` — возвращает поток, в котором отсутствуют первые `n` элементов.
- `Stream<T> limit(long maxSize)` — оставляет в потоке только `maxSize` элементов.
- `Stream<T> takeWhile(Predicate<? super T> predicate)` — берёт элементы из потока, пока выполняется предикат.
- `Stream<T> dropWhile(Predicate<? super T> predicate)` — пропускает элементы, которые соответствуют условию в `predicate`, пока не попадет элемент, который не соответствует условию.
- `Stream<T> concat(Stream<? extends T> a, Stream<? extends T> b)` — объединяет два потока.
- `Stream<T> distinct()` — возвращает поток, в котором имеются только уникальные данные с типом `T`.

Терминальные операции

Терминальные операции обрабатывают элементы и завершают работу стрима, так что терминальный оператор в цепочке может быть только один. Также называются `eager`, до их вызова не применяются `lazy` операции.

Вот некоторые из них:

- `long count()` — возвращает количество элементов в потоке.
- `void forEach(Consumer<? super T> action)` — для каждого элемента выполняется действие `action`.
- `boolean allMatch(Predicate<? super T> predicate)` — возвращает `true`, если все элементы потока удовлетворяют условию в предикате.
- `boolean anyMatch(Predicate<? super T> predicate)` — возвращает `true`, если хоть один элемент потока удовлетворяет условию в предикате.
- `boolean noneMatch(Predicate<? super T> predicate)` — возвращает `true`, если ни один из элементов в потоке не удовлетворяет условию в предикате.

- `Optional<T> findFirst()` – возвращает первый элемент из потока.
- `Optional<T> findAny()` – возвращает первый попавшийся элемент из потока. Это не то же самое, что `findFirst`, по крайней мере при параллелизме: он возвращает первый же результат, найденный любым из свободных ядер процессора, не дожидаясь завершения обработки "более ранних" элементов.
- `Object[] toArray()` – возвращает массив из элементов потока.
- `<R,A> R collect(Collector<? super T,A,R> collector)` – добавляет элементы в неизменяемый контейнер с типом `R`. `T` представляет тип данных из вызывающего потока, а `A` - тип данных в контейнере.

Про коллекторы стоит рассказать подробнее (тем более это требует билет).

Простые коллекторы

Вернёмся к методу `<R,A> R collect(Collector<? super T,A,R> collector)`. Здесь параметр `R` представляет тип результата метода, параметр `T` - тип элемента в потоке, а параметр `A` - тип промежуточных накапливаемых данных. В итоге параметр `collector` представляет функцию преобразования потока в коллекцию. Эта функция представляет объект `Collector`, который определен в пакете `java.util.stream`.

В классе `Collectors` Java предоставляет несколько готовых коллекторов:

- `toList()`: преобразование к типу `List`
- `toSet()`: преобразование к типу `Set`
- `toMap()`: преобразование к типу `Map`

`toList` и `toSet` вызываются без аргументов:

```
List<Integer> list = IntStream.range(0, 9).boxed().collect(Collectors.toList());
```

`toMap()` так или иначе требует получать ключи и значения с помощью лямбд (ещё есть перегрузка с бинарным оператором, и перегрузка с `supplier`, лучше глянуть [документацию](#)).

Рассмотрим несколько примеров использования, чтобы покрыть эти кейсы.

```
import java.util.*;
import java.util.stream.*;

record User(String email, String name) {}

public class Main {
    public static void main(String args[]) {
```

```

Map<String, String> map = Stream.of(
    new User("yasos.biba@phystech.edu", "MemelordSixSeven"),
    new User("somemail@rambler.ru", "TestUser13")
)
    .collect(Collectors.toMap(u -> u.email(), u -> u.name()));
}
}

```

Здесь `record` – класс с приватными полями и геттерами, это небольшой QoL из Java 17 для создания DTO и т.п. В качестве лямбд `keyMapper` и `valueMapper` применяются соответствующие геттеры.

```

Map<Integer, Integer> integerMap = Stream.of(1, 1, 1, 2, 3, 3, 3, 3, 3)
    .collect(Collectors.toMap(e -> e, e -> 1, Integer::sum));

```

Здесь считается количество вхождений числа в список. Если значение по ключу уже есть в мапе, то третий аргумент (`BinaryOperator<U> mergeFunction`) применяет бинарный оператор к результату `valueMapper` и значению в мапе.

```

SortedMap<Integer, Integer> integerMap = Stream.of(1, 1, 1, 2, 3, 3, 3, 3, 3)
    .collect(Collectors.toMap(e -> e, e -> 1, Integer::sum, TreeMap::new));

```

Четвёртый аргумент позволяет выбрать конкретную получаемую реализацию `Map`. По умолчанию, это `HashMap`. В этом примере мы получили `TreeMap`. Это полезно, если нам нужно получить именно `SortedMap` или `NavigableMap`, где ключи отсортированы.

Сложные коллекторы

Сложные коллекторы – это, по сути, композиция коллекторов. Стандартные реализации позволяют делать ряд агрегирующих/редуцирующих операций.

Коллектор `Collectors.groupingBy()` группирует данные по некоторому признаку:

```

record Book(String author, String title, String genre, int pageCount) {}

List<Book> books = List.of(
    new Book("Пушкин", "Евгений Онегин", "Роман", 352),
    new Book("Лермонтов", "Герой нашего времени", "Роман", 224),
    new Book("Гоголь", "Мёртвые души", "Поэма", 560),
    new Book("Гоголь", "Шинель", "Повесть", 64)
);

Map<String, List<Book>> byGenre = books.stream()

```

```
.collect(groupingBy(Book::getGenre));  
// byGenre = {Поэма=['Мёртвые души'], Роман=['Евгений Онегин', 'Герой нашего  
времени'], Повесть=['Шинель']}
```

Также может принимать другой коллектор как аргумент, вот полезные примеры:

- `Collectors.counting()` применяется в `Collectors.groupingBy()` для вычисления количества элементов в каждой группе.
- Коллекторы `Collectors.summingInt()`, `Collectors.summingLong()`, `Collectors.summingDouble()` применяются для подсчёта суммы.
- Коллекторы `Collectors.averagingInt()`, `Collectors.averagingLong()`, `Collectors.averagingDouble()` применяются для подсчёта среднего.
- `Collectors.maxBy` и `Collectors.minBy` применяются для подсчета минимального и максимального значения в каждой группе. В качестве параметра эти методы принимают функцию компаратора, которая нужна для сравнения значений.
- Методы `summarizingInt()` / `summarizingLong()` / `summarizingDouble()` позволяют объединить в набор значения соответствующих типов. Результатом будет объект типа `IntSummaryStatistics` / `LongSummaryStatistics` / `DoubleSummaryStatistics`, имеющий методы `getAverage()`, `getMin()`, `getMax()`, `getCount()`, `getSum()`, ...
- Сгруппированные элементы можно преобразовать, применив коллектор `Collectors.mapping`
- Сгруппированные элементы можно отфильтровать, применив дополнительный коллектор `Collectors.filtering`

И несколько примеров:

```
Map<String, Integer> totalLengthByGenre = books.stream()  
    .collect(groupingBy(Book::getGenre,  
        summingInt(Book::getPageCount)));  
  
// totalLengthByGenre = {Поэма=560, Роман=576, Повесть=64}  
  
Map<String, Double> averageLength = books.stream()  
    .collect(groupingBy(Book::getGenre,  
        averagingInt(Book::getPageCount)));  
  
// averageLength = {Поэма=560.0, Роман=288.0, Повесть=64.0}  
  
Map<String, IntSummaryStatistics> statisticsByGenre = books.stream()  
    .collect(groupingBy(Book::getGenre,
```

```

        summarizingInt(Book::getPageCount())));
/*
statisticsByGenre = {
    Поэма=IntSummaryStatistics{
        count=1, sum=560, min=560, average=560.000000, max=560},
    Роман=IntSummaryStatistics{
        count=2, sum=576, min=224, average=288.000000, max=352},
    Повесть=IntSummaryStatistics{
        count=1, sum=64, min=64, average=64.000000, max=64}
}
*/

Map<String, Optional<Book>> maxLengthByAuthor = books.stream()
    .collect(groupingBy(Book::getAuthor,
        maxBy(Comparator.comparingInt(Book::getPageCount()))));

// maxLengthByAuthor = {Пушкин=Optional['Евгений Онегин'], Гоголь=Optional['Мёртвые
души'], Лермонтов=Optional['Герой нашего времени']}

Map<String, Set<String>> genresByAuthor = books.stream()
    .collect(groupingBy(Book::getAuthor,
        mapping(Book::getGenre, toSet())));

// genresByAuthor = {Пушкин=[Роман], Гоголь=[Поэма, Повесть], Лермонтов=[Роман]}

Map<String, Set<Book>> shortBooks = books.stream()
    .collect(groupingBy(Book::getAuthor,
        filtering(b -> b.getPageCount() < 100, toSet())));

// shortBooks = {Пушкин=[], Гоголь=['Шинель'], Лермонтов=[]}

```

(необязательно) Параллелизм в StreamAPI

Кроме последовательных потоков Stream API поддерживает параллельные потоки. Распараллеливание потоков позволяет задействовать несколько ядер процессора и тем самым может повысить производительность и ускорить вычисления.

Сделать из последовательного потока параллельный можно сделать методом `parallel()`:

```

Stream.of(1, 2, 3) .parallel().map(e -> e * 2) .forEach(System.out::println);
// 4 6 2 или какая-то другая перестановка

```

Также можно сразу создать параллельный поток из коллекции методом `parallelStream()`:

```
List.of(1, 2, 3) .parallelStream().map(e -> e * 2) .forEach(System.out::println);  
// 4 6 2 или какая-то другая перестановка
```

Stream API также позволяет собрать параллельные потоки в один с помощью метода `sequential()` :

```
List.of(1, 2, 3)  
    .parallelStream()  
    .map(e -> e * 2)  
    .sequential()  
    .forEach(System.out::println);  
  
// 2 4 6
```

Методы `parallel()` и `sequential()` – промежуточные операции, они лишь указывают терминальной операции, как обрабатывать элементы, и сами не создают параллельных или последовательных потоков, вот как они выглядят в исходниках:

```
public final S parallel() {  
    sourceStage.parallel = true;  
    return (S) this;  
}  
  
public final S sequential() {  
    sourceStage.parallel = false;  
    return (S) this;  
}
```

Как правило, элементы передаются в поток в том же порядке, в котором они определены в источнике данных. При работе с параллельными потоками система сохраняет порядок следования элементов. Исключение составляет метод `forEach()` , который может выводить элементы в произвольном порядке. И чтобы сохранить порядок следования, необходимо применять метод `forEachOrdered` .

```
List.of(1, 2, 3)  
    .parallelStream()  
    .map(e -> e * 2)  
    .forEachOrdered(System.out::println);  
  
// 2 4 6
```

Сохранение порядка в параллельных потоках увеличивает издержки при выполнении. Но если нам порядок не важен, то мы можем отключить его сохранение и тем самым увеличить производительность, используя метод `unordered()`.

8. Аннотации. Объявление и применение аннотаций. Мета-аннотации `@Target`, `@Retention`. Параметры аннотаций.

Аннотации – это специальные метаданные, которые встраиваются в исходный код программы и используются для описания классов, методов, полей и других элементов. Они не изменяют поведение программы напрямую (обычно...), но могут быть использованы:

- компилятором (проверки, предупреждения);
- инструментами генерации кода и документации;
- фреймворками и библиотеками;
- во время выполнения программы через reflection.

Таким образом, аннотации позволяют декларативно описывать намерения разработчика, не засоряя код логикой.

Каждая аннотация имеет два обязательных параметра:

- `@Target` – тип объекта, над которым она указывается.
 - `ElementType.FIELD` – поле класса.
 - `ElementType.TYPE` – указывается над классом.
 - `ElementType.METHOD` – указывается над методом.
 - `ElementType.PARAMETER` – параметр метода: (`public void foo(@MyAwesomeAnnotation String param)`).
 - `ElementType.LOCAL_VARIABLE` – локальная переменная.
 - `ElementType.CONSTRUCTOR` – конструктор класса.
 - `ElementType.ANNOTATION_TYPE` – другая аннотация.
 - `ElementType.PACKAGE` – описание пакета `package`.
- `@Retention` – тип хранения, т.е. жизненный цикл аннотации: будет она присутствовать только в исходном коде, в скомпилированном файле, или она будет также видна и в процессе выполнения. Выбор нужного типа зависит от того, как вы хотите использовать аннотацию, например, генерировать что-то побочное из исходных кодов, или в процессе выполнения стучаться к классу через reflection.
 - `RetentionPolicy.SOURCE` – аннотация используется только при написании кода и игнорируется компилятором (т.е. не сохраняется после компиляции). Обычно

используется для каких-либо препроцессоров (условно), либо указаний компилятору.

- `RetentionPolicy.CLASS` – аннотация сохраняется после компиляции, однако игнорируется JVM (т.е. не может быть использована во время выполнения). Обычно используется для каких-либо сторонних сервисов, подгружающих ваш код в качестве plug-in приложения.
- `RetentionPolicy.RUNTIME` – аннотация которая сохраняется после компиляции и подгружается JVM (т.е. может использоваться во время выполнения самой программы). Используется в качестве меток в коде, которые напрямую влияют на ход выполнения программы.

Примеры аннотаций из стандартной библиотеки (см. [документацию](#)):

`@Override` – Retention: SOURCE, Target: Method. Показывает, что метод, над которым она написана унаследован у родительского класса. Практическая польза: видно, где переопределяется метод. Если аннотация стоит, но данного метода нет в родительских классах, то компилятор выдаст ошибку.

`@Deprecated` – Retention: RUNTIME, Target: CONSTRUCTOR, FIELD, LOCAL_VARIABLE, METHOD, PACKAGE, PARAMETER, TYPE. Данная аннотация указывает на методы, классы или переменные, которые являются "устаревшими" и могут быть убраны в последующих версиях продукта.

`@SuppressWarnings` – Retention: SOURCE, Target: TYPE, FIELD, METHOD, PARAMETER, CONSTRUCTOR, LOCAL_VARIABLE. Данная аннотация отключает вывод предупреждений компилятора, которые касаются элемента над которым она указана.

`@Retention` и `@Target` – Retention: RUNTIME, Target: ANNOTATION_TYPE. Задают Retention и Target для аннотаций, в том числе кастомных и для самих себя.

`@Documented` – Retention: RUNTIME, Target: ANNOTATION_TYPE. Указывает, что помеченная таким образом аннотация должна быть добавлена в javadoc поля/метода и т.д.

`@Inherited` – Retention: RUNTIME, Target: ANNOTATION_TYPE. Помечает аннотацию, которая будет унаследована потомком класса, отмеченного такой аннотацией.

`@FunctionalInterface` – Retention: RUNTIME, Target: TYPE. уже встречалась ранее. Это интерфейсы с единственным абстрактным методом, нужны для лямбд.

`@Native` – Retention: SOURCE, Target: FIELD. указывает, что константы в интерфейсе предназначены для статического анализа.

`@Repeatable` – Retention: `RUNTIME`, Target: `ANNOTATION_TYPE`. позволяет одной аннотации повторяться несколько раз над одним элементом.

Создание и применение

Аннотация определяется описанием с ключевым словом `@interface` и может включать в себя несколько полей (параметров), описываемых как будто функции (но это просто поля с конкретным значением). В качестве параметров могут быть использованы только примитивы, `String`, `Enum`.

Вот пример аннотации `@NotEmpty` и работа с ней с помощью `Java Reflection`:

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;

public class Main {
    public static void main(String[] args) {
        DataClass d1 = new DataClass("Good one");
        DataClass d2 = new DataClass("");
        DataClass d3 = new DataClass();

        try {
            NotEmptyValidator.checkConstraints(d1);
        } catch (IllegalStateException e) {
            System.out.println(d1.toString() + ": " + e.getMessage());
        }

        try {
            NotEmptyValidator.checkConstraints(d2);
        } catch (IllegalStateException e) {
            System.out.println(d2.toString() + ": " + e.getMessage());
        }

        try {
            NotEmptyValidator.checkConstraints(d3);
        } catch (IllegalStateException e) {
            System.out.println(d3.toString() + ": " + e.getMessage());
        }
    }
}

// === DTO example ===
```

```

class DataClass {
    @NotEmpty
    String field;
    int filler = 15;

    public DataClass() {}

    public DataClass(String field) {
        this.field = field;
    }
}

// === Annotation ===

@Retention(value = RetentionPolicy.RUNTIME)
@Target(value = ElementType.FIELD)
@interface NotEmpty {
    String message() default "Field must not be empty";
}

// === Validator ===

class NotEmptyValidator {
    public static void checkConstraints(Object o) throws IllegalStateException{
        for (Field f : o.getClass().getDeclaredFields()) {
            if (f.isAnnotationPresent(NotEmpty.class)) {
                f.setAccessible(true);
                try {
                    Object value = f.get(o);
                    if (value == null || value.toString().isEmpty()) {
                        throw new
IllegalStateException(f.getAnnotation(NotEmpty.class).message());
                    }
                } catch (IllegalAccessException e) {}
            }
        }
    }
}

```

Сами по себе аннотации бесполезны, но с ними можно работать, используя *Reflection API*. Благодаря этому, аннотации играют ключевую роль в фреймворке *Spring Boot*, где с их помощью описывается конфигурация приложения, внедрение зависимостей и обработка HTTP-запросов. Такие аннотации, как `@Component`, `@Autowired`, `@Service`, `@Controller`, позволяют отказаться от ручкой написанной конфигурации и управлять поведением

приложения декларативно. Фактически, Spring Boot – это пример того, как аннотации напрямую влияют на жизненный цикл объектов и логику выполнения программы.

9. Основы многопоточности. Потоки (threads). Создание и завершение потоков. Проблема race condition. Мьютексы и их использование.

Напоминание из АКОСа:

Процесс – запущенный исполняемый код, имеющий атрибуты, которые навесила на него операционная система, некоторая сущность в ядре ОС.

Треды – это подпроцессы, у которых почти все данные общие с исходным процессом. Подробнее: динамическая и статическая памяти, файловые дескрипторы, файловое пространство (например cwd) у тредов разделяемые, стек у каждого тредов свой.

Когда запускается программа, начинает работать главный поток этой программы, в котором и выполняется метод `main()`. От этого главного потока порождаются все остальные дочерние потоки.

В языке Java за работу с потоками отвечает класс `Thread`. Для создания потока необходимо создать экземпляр этого класса.

Основные способы создать поток с какой-то `Runnable` задачей это:

- Отнаследоваться от класса `Thread` и переопределить метод `run()`.
- Передать в конструктор `Thread(Runnable r)` лямбду вида `() -> { /* void */ }`
- Реализовать интерфейс `Runnable` в своём классе и передать в конструктор.

```
class Task implements Runnable {
    @Override
    void run() {
        System.out.println("What a waste of a syscall");
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t = new Thread(new Task());
        // ...
    }
}
```

```
}  
}
```

Также в конструкторе можно задать имя потока (по умолчанию, "Thread" + n, где n – нек. целое число) и его ThreadGroup (если null, то такая же, как у текущего потока).

Потоки в Java в любой момент времени находятся в одном из 6 состояний:

- NEW : поток создан, но еще не запущен
- RUNNABLE : поток выполняется
- BLOCKED : поток заблокирован в ожидании блокировки монитора
- WAITING : поток неопределенно долго ожидает, пока другой поток выполнит определенное действие
- TIMED_WAITING : поток, ожидающий выполнения действия другим потоком в течение заданного времени
- TERMINATED : поток завершил выполнение

Эти состояния представляют собой состояния виртуальной машины и не отражают состояния потоков операционной системы.

Также потоки имеют приоритет – число от 1 до 10. Для работы с ними предоставлено 3 константы: MIN_PRIORITY (1), NORM_PRIORITY (5), MAX_PRIORITY (10), а также метод `setPriority()`. Всякий раз, когда у планировщика потоков платформы есть возможность выбрать новый поток, он отдает предпочтение потокам с более высоким приоритетом.

Запуск и завершение потока

Для запуска потока используется метод `start()`, переводящий его в состояние RUNNABLE. Однако в реальности такой поток еще не выполняется: он может быть запущен, а может и не запущен. Планировщик потоков решает, предоставить ли потоку время для выполнения. После того, как поток запущен, он может периодически приостанавливаться, чтобы дать другим потокам возможность выполниться. Однако точные детали планирования потоков зависят от операционной системы и от природы потока.

Касаясь завершения/жизненного цикла стоит упомянуть следующие методы:

Метод `Thread.join()` используется для ожидания завершения выполнения другого потока. Когда вызывается метод `join()` на определенном потоке, текущий поток будет приостановлен до тех пор, пока указанный поток не завершится.

Статический метод `Thread.sleep()` приостанавливает текущий поток на определенный период времени. Этот метод имеет ряд версий:

```
static void sleep(long millis)
static void sleep(long millis, int nanos)
static void sleep(Duration duration)
```

Класс `Thread` предоставляет метод `stop()`, однако он почти сразу был объявлен как `@Deprecated`. Дело в том, что `stop()` освобождает все мониторы (locks), которые были заняты данным потоком. Если в процессе выполнения операции объект, доступ к которому был ограничен с помощью монитора, находится в неконсистентном состоянии, есть вероятность, что другие потоки так же увидят его в "поврежденном" виде. Чтобы избежать таких ошибок, метод `stop()` не рекомендован к использованию.

Вместо этого в Java есть механизм *прерываний* – метод `interrupt()`. Данный метод установит флаг прерывания потока (прекращения работы), который можно проверить методом `isInterrupted()` и указать логику его обработки и завершения работы потока. Если же поток был заблокирован, находился в ожидании (`wait`, `sleep`, `join`), то будет сброшен флаг прерывания и выброшено исключение `InterruptedException`. (Если проверяется условие `isInterrupted()` в цикле, чтобы по команде завершить поток, то просто снова поднимаете флаг в `catch` блоке: `Thread.currentThread().interrupt()`).

```
class PrimeGenerator implements Runnable {
    private final BlockingQueue<BigInteger> primes;

    public PrimeGenerator(BlockingQueue<BigInteger> primes) {
        this.primes = primes;
    }

    @Override
    public void run() {
        BigInteger p = BigInteger.ONE;
        try {
            while (!Thread.currentThread().isInterrupted()) {
                p = p.nextProbablePrime();
                primes.put(p);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Race Condition. Синхронизация.

И снова обратимся к курсу АКОСа:

Race condition (состояние гонки) – это ситуация, которая возникает в многозадачных или многопоточных программных системах, когда результаты выполнения программы зависят от того, в какой последовательности выполняются инструкции или операции в различных потоках или процессах.

Состояние гонки возникает, когда несколько потоков или процессов одновременно пытаются получить доступ к одним и тем же ресурсам (например, переменным памяти, файлам или устройствам ввода-вывода) без синхронизации.

Пример состояния гонки: два потока пытаются изменить одну и ту же переменную. Если эти изменения не синхронизированы правильно, результаты могут быть непредсказуемыми, в зависимости от того, какой поток завершит операцию первым.

Состояние гонки может привести к непредсказуемому поведению программы или к ошибкам в её выполнении.

Простейший способ в Java борьбы с race condition – использование `synchronized` блоков.

`synchronized`-блок, синхронизированная секция или т.п. представляет блок кода, к которому одномоментно имеет доступ только один поток. Он имеет следующий вид:

```
synchronized (object) {  
    // some shady actions  
}
```

При создании синхронизированного блока кода после оператора `synchronized` идет объект-заглушка: `synchronized(object)`. Причем в качестве объекта может использоваться только объект какого-нибудь класса, но не примитивного типа. Затем идет блок кода, в которую помещается критическая секция – код, к которому в один момент времени имеет доступ только один поток.

Каждый объект в Java имеет ассоциированный с ним монитор. Монитор представляет своего рода инструмент для управления доступа к объекту. Когда выполнение кода доходит до оператора `synchronized`, монитор объекта блокируется, и на время его блокировки монопольный доступ к блоку кода имеет только один поток, который и произвел блокировку. После окончания работы блока кода, монитор объекта освобождается и становится доступным для других потоков. После освобождения монитора его захватывает другой поток, а все остальные потоки продолжают ожидать его освобождения.

Абсолютно аналогичный способ – использование оператора `synchronized` на методе с потокобезопасным кодом. Здесь тот же принцип работы, монитором выступает монитор объекта `this`.

```
class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }
}
```

Рассмотрим такую задачу, у нас N потоков, угадывающих случайные числа от 0 до 999. Поток получает очки, если угадал число первым. И нам запрещено использовать потокобезопасные коллекции, атомики и прочий инструментарий. С помощью только оператора `synchronized` мы можем это написать так:

```
package threads;

import java.util.ArrayList;
import java.util.List;
import java.util.Random;

public class Main {
    private static final int PLAYERS = 8;

    public static void main(String[] args) {
        List<GuesserThread> threads = new ArrayList<>();

        for (int i = 0; i < PLAYERS; i++) {
            GuesserThread t = new GuesserThread(42 + i);
            threads.add(t);
            t.start();
        }

        for (GuesserThread t : threads) {
            try {
                t.join();
            } catch (InterruptedException e) {}
        }

        for (GuesserThread t : threads) {
            t.printScore();
        }
    }
}

class NumberBoard {
```

```

private final boolean[] board;
private int counter = 0;

public NumberBoard(int N) {
    board = new boolean[N];
}

public synchronized boolean isFinished() {
    return counter == board.length;
}

public synchronized boolean tryGuess(int n) {
    if (counter == board.length) return false;

    if (!board[n]) {
        board[n] = true;
        counter++;
        return true;
    }

    return false;
}
}

class GuesserThread extends Thread {
    private static final int NUMBERS_BOUND = 1000;
    private static final NumberBoard board = new NumberBoard(NUMBERS_BOUND);

    private int score = 0;
    private Random random;

    public GuesserThread(long seed) {
        random = new Random(seed);
    }

    @Override
    public void run() {
        while (!board.isFinished()) {
            if (board.tryGuess(random.nextInt(NUMBERS_BOUND))) {
                score++;
            }
        }
    }

    public void printScore() {
        System.out.println(super.getName() + " :: " + score + " pts.");
    }
}

```

```
}  
}
```

Читатели, знакомые с многопоточным программированием, легко заметят, что данный код далёк от оптимального: используется один общий мьютекс, отсутствует справедливость между потоками, применяется активное ожидание, а масштабируемость решения низкая. Тем не менее, эти недостатки оставлены намеренно. Пример призван только и только показать проблему `race condition` и её устранение с помощью `synchronized`, не выходя за рамки базовых механизмов потоков и мьютексов.

Повторение / изучение всех остальных, не требуемых в билете, аспектов многопоточки в Java оставляется добровольным (~~лучше читайте хабр, моя селективная копияста-кринж~~).

10. Spring Framework. Чем отличается управление объектами через `new` от управления через Inversion of Control. Какие способы внедрения зависимостей поддерживаются. Что такое Spring Bean и какие аннотации используются для создания бинов. Скоупы бинов.

Spring – это общее название для группы фреймворков, реализующих разнообразную функциональность: безопасность, web-приложения, инструменты для доступа к данным, интеграции между сервисами, обработки больших объемов данных.

IoC, DI. Способы внедрения зависимостей.

Один из основных принципов Spring – инверсия управления (IoC, Inversion of Control). Инверсия управления – это то, что по сути различает фреймворк и библиотеку. Библиотека – это набор функций/классов, которые вы можете вызывать. Каждый вызов выполняет некоторую работу и возвращает управление обратно пользователю. Фреймворк же воплощает в себе некоторый абстрактный дизайн со встроенным поведением. Для того чтобы использовать его, вы должны добавить свой код в различных местах фреймворка, а код фреймворка впоследствии будет вызывать ваш код. Ваш код получает управление на какое-то время, делает свою работу и возвращает управление обратно фреймворку. В этом и заключается инверсия управления.

Spring использует IoC для "сборки" приложения из маленьких кусков. Такой паттерн называется внедрением зависимостей, он же Dependency Injection (DI). Основная идея DI заключается в том, что за выбор нужной имплементации отвечает отдельная сущность – "сборщик" (Assembler), он же внедряет конкретные реализации интерфейсов.

Добавлять зависимость в класс можно разными способами. Чтобы указать, что в каком-то конкретном месте нужно внедрить зависимость, используется аннотация `@Autowired`.

Constructor Injection

Если в классе есть конструктор, то можно внедрить зависимость через конструктор. При создании класса контейнер Spring вызовет конструктор и передаст зависимость в качестве аргумента конструктора.

Это рекомендуемый способ. Он позволяет делать поля `final`, гарантирует, что объект не будет создан без нужных зависимостей, и упрощает юнит-тестирование (можно просто передать mock-объект в конструктор без поднятия контекста Spring).

```
@Component
public class CourseLister {
    private final CourseFinder finder;

    @Autowired
    public CourseLister(CourseFinder finder) {
        this.finder = finder;
    }

    public List<Course> coursesByAuthor(String name) {
        List<Course> allCourses = finder.findAll();
        return allCourses
            .stream()
            .filter(course -> course.getAuthor().equals(name))
            .collect(Collectors.toList());
    }
}
```

Начиная со Spring 4.3, если в классе только один конструктор, аннотацию `@Autowired` над ним можно не ставить – Spring найдет его автоматически.

Setter Injection

Если в классе задан сеттер, то зависимость можно внедрить и через него. Тогда при создании экземпляра класса контейнер вызовет конструктор без аргументов, а потом сеттер, чтобы внедрить зависимость во только что созданный бин.

```
@Component
public class CourseLister {
    private CourseFinder finder;
```

```

    @Autowired
    public void setFinder(CourseFinder finder) {
        this.finder = finder;
    }
}

```

Так же, как в предыдущем случае, аннотацию `@Autowired` перед сеттером можно опустить.

Используется, когда зависимость является необязательной или может быть изменена во время работы приложения.

Field Injection

Можно обойтись вообще без сеттера:

```

public class CourseLister {
    @Autowired
    private CourseFinder finder;
}

```

И внедрение зависимости все равно произойдет. Несмотря на то, что тут нет ни конструктора, ни сеттера, а поле *engine* имеет модификатор *private*. Это возможно, потому что под капотом фреймворк использует рефлексия для создания бинов.

Из-за этого класс станет сложно тестировать: понадобится Spring контекст. Также будет нельзя объявить поле как `final`.

Больше узнать про это можно в данной статье на Хабре:

<https://habr.com/ru/articles/983344/>

@Qualifier и @Primary

Представим ситуацию, когда у нас несколько бинов реализуют какой-то интерфейс. В Spring контексте оказывается несколько объектов одного типа, и при старте приложения возникнет исключение, поскольку Spring не сможет определить, какой именно из этих бинов нужно заинжектировать.

```

@Component
public class UserServiceImpl implements UserService {
    ...
}

@Component

```

```

public class AuditUserService implements UserService {
    @Autowired
    private UserService origin;
    ...
}

@Component
public class EmailUserService implements UserService {
    @Autowired
    private UserService origin;
    ...
}

```

Для решения этой проблемы есть две аннотации:

@Qualifier над полем (и под **@Autowired**) позволяет уточнить какой бин нужно внедрить. Для этого в **@Component** мы указываем имя как параметр. Если этого не делать, Spring автоматически генерирует camelCase строчку.

```

@Component("userServiceImpl")
public class UserServiceImpl implements UserService {
    ...
}

@Component("auditUserService")
public class AuditUserService implements UserService {
    @Autowired
    @Qualifier("userServiceImpl")
    private UserService origin;
    ...
}

@Component("emailUserService")
public class EmailUserService implements UserService {
    @Autowired
    @Qualifier("auditUserService")
    private UserService origin;
    ...
}

```

Аннотация **@Primary** указывается над бином. Этот бин будет выбираться по умолчанию при инжектировании:

```

@Component
@Primary
public class UserServiceImpl implements UserService {
    ...
}

@Component
public class AuditUserService implements UserService {
    @Autowired
    private UserService origin;
    ...
}

@Component
public class EmailUserService implements UserService {
    @Autowired
    private UserService origin;
    ...
}

```

Также отметим, что `@Qualifier` имеет более высокий приоритет, чем `@Primary`. Даже если есть `@Primary`, `@Qualifier` выберет указанный компонент.

Чрезмерное использование этих аннотаций может привести к коду, который сложнее рефакторить или масштабировать, но при разумном использовании он может быть практичным инструментом для поддержания ясности и разрешения конфликтов в сложных приложениях на Spring.

Что такое Spring Bean и какие аннотации используются для создания бинов.

Когда стартует Spring Boot приложение, оно начинает сканировать все классы вверх по иерархии в вашем проекте. То есть если ваш `main` находится в пакете `com.mycompany`, то будут просканированы все классы, у которых пакет начинается с `com.mycompany`. Поменять корень скана можно с помощью параметра `scanBasePackages`:

```

package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication(scanBasePackages = "com.example")
public class DemoApplication {
    public static void main(String[] args) {

```

```
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

(Но проще закинуть `main` в корень проекта).

Когда Spring видит класс, на котором стоит аннотация `@Component`, он понимает, что он является его частью. Так что он создает экземпляр этого класса (в коде вы бы это сделали через `new`) и добавляет его в Spring Context. Если по-простому, Spring Context – это `HashMap`, который содержит экземпляры всех классов, которые Spring воспринял как `beans` (так называются те классы, которые просканированы Spring, а их экземпляры добавлены в контекст). Бины хранятся в качестве значения `HashMap`. Ключом же выступает название бина. По умолчанию это имя класса с маленькой буквы. Если в вашем приложении несколько классов с одинаковыми именами, то Spring может поменять стратегию наименования по умолчанию, чтобы сохранить уникальность.

Все бины по умолчанию синглтоны. То есть если вы инжектируете один и тот же класс в разных местах, то будете получать один и тот же объект, который создал для вас Spring. Это поведение логично во многих сценариях. Тем не менее вам следует помнить, что если вы хотите внутри класса, который является Spring Bean, хранить и менять какое-то состояние, то вам нужно соблюдать правила потокобезопасного доступа. То есть использовать потокобезопасные коллекции, атомики, `locks` и так далее.

Создание экземпляров класса происходит с помощью механизма `Reflection API`. Дело в том, что `class` в Java тоже является объектом, у которого есть методы. Получив его, можно создавать экземпляры этого объекта.

Помимо аннотации `@Component` для объявления бинов используются аннотации `@Service` и `@Repository`. На текущий момент технической разницы между `@Service` и `@Component` нет и они используются скорее для зрительного разделения частей приложения.

`@Repository` – специализированный компонент для уровня доступа к данным. Включает механизм `PersistenceExceptionTranslationPostProcessor`, который перехватывает специфичные для БД исключения (например, `SQLException`) и оборачивает их в иерархию исключений Spring (`DataAccessException`). Работает через механизмы AOP (проксирование).

Также бывают ситуации, когда мы явно хотим создавать наши бины через `new`. Например, когда создание объекта требует сложной логики, настройки сторонних библиотек или когда невозможно изменить исходный код класса. Для этого используются аннотации `@Configuration` и `@Bean`.

- `@Configuration` указывает, что класс является источником определений бинов.

- `@Bean` аннотация над методом внутри конфигурационного класса. Результат выполнения метода регистрируется как бин в контексте.

Пример:

```
@Configuration
public class RestTemplateConfiguration {
    @Bean
    public RestTemplate restTemplate(@Value("${author-registry-service.base.url}")
    String baseUrl,
                                   @Value("${author-registry-
service.timeout.seconds}") int secondsTimeout) {
        Duration timeout = Duration.ofSeconds(secondsTimeout);
        return new RestTemplateBuilder()
            .setConnectTimeout(timeout)
            .setReadTimeout(timeout)
            .rootUri(baseUrl)
            .build();
    }
}
```

Не следует дублировать аннотации. Если класс помечен `@Component`, и он же создается через `@Bean`, в контексте возникнет коллизия (два бина одного типа), что приведет к `NoUniqueBeanDefinitionException` при попытке внедрения через `@Autowired`.

@Scope

Аннотация `@Scope` в Spring Boot (и Spring Framework в целом) определяет жизненный цикл бина – то, как долго объект живет, когда он создается и как часто переиспользуется.

Основные скоупы:

- `@Scope("singleton")` – это скоуп по умолчанию. Когда бин помечен этим скоупом (или не помечен вообще), Spring создаёт только один экземпляр бина на весь `ApplicationContext`. Этот бин будет использоваться повторно каждый раз, когда он запрашивается.
- `@Scope("prototype")` – скоуп для случаев, когда вам нужен новый экземпляр бина при каждом запросе. Это полезно, если бин содержит состояние, которое меняется в зависимости от бизнес-логики.
- `@Scope("request")` используется в веб-приложениях. На каждый новый HTTP-запрос создаётся отдельный экземпляр бина.
- `@Scope("session")` – бин, уникальный для сессии пользователя. Используется в веб-приложениях.

11. Spring MVC. Отличие @Controller от @RestController . МAPPING HTTP-запросов (@GetMapping , @PostMapping и т.д). Обработка исключений с помощью @ControllerAdvice . Интерсепторы. Валидация объектов (@NotBlank , @Email и т.д).

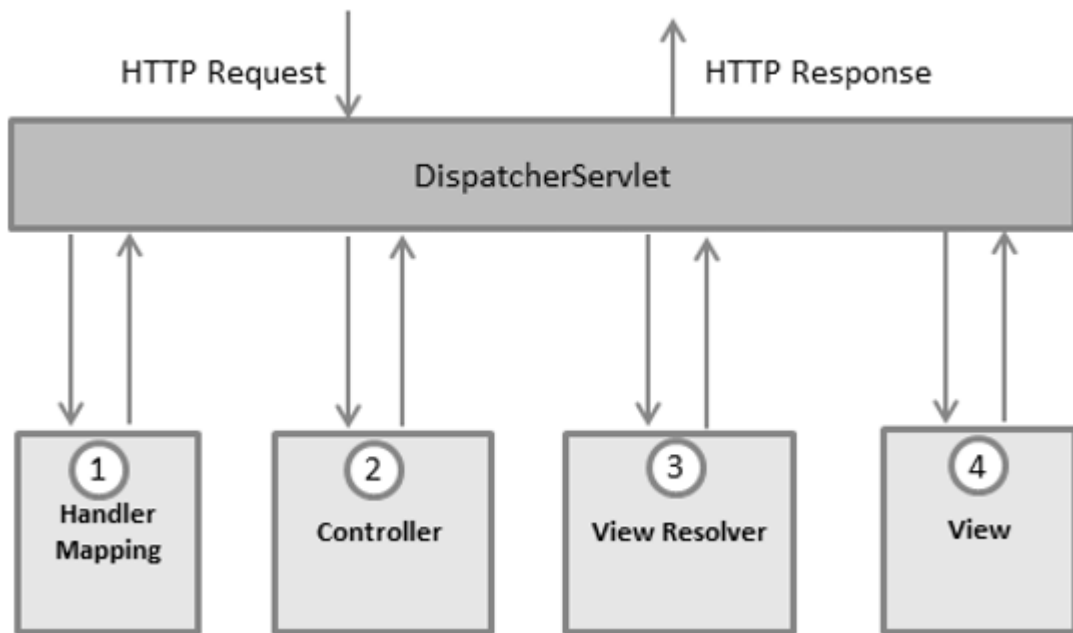
Spring MVC

Фреймворк Spring Web MVC (часто упоминается как "Spring MVC") – это фреймворк для веб-приложений, построенный на основе Spring Core и Java Servlet API. Он использует архитектуру Model - View - Controller (отсюда и название).

Главной целью MVC является разделение объектов, бизнес-логики и внешнего вида приложения. Все эти компоненты слабо связаны между собой и при желании мы можем изменить, например, внешний вид приложения, не внося существенные изменения в остальные два компонента.

- Блок Model инкапсулирует данные приложения. На практике это POJO-классы (Plain Old Java Objects – Простые старые Java-объекты).
- Модуль представления (View) отвечает за вывод данных пользователю. Обычно это JSP файл, который может быть опознан и интерпретирован браузером на пользовательской машине.
- Контроллер (Controller) отвечает за обработку запросов пользователей и передачу данных модулю View для обработки.

В основе Spring MVC Framework лежит DispatcherServlet, задача которого – обработка всех HTTP запросов и ответов.



После получения HTTP-запроса DispatcherServlet (далее – DS) выполняет следующие действия:

1. **Поиск контроллера:** Получив HTTP-запрос, DispatcherServlet обращается к **HandlerMapping**. Тот находит подходящий контроллер и возвращает "цепочку" выполнения (Handler Execution Chain).
2. **Выполнение логики:** DS передает запрос в **HandlerAdapter**, который вызывает метод контроллера. Контроллер выполняет бизнес-логику и возвращает объект **ModelAndView**, содержащий данные (модель) и логическое имя вида (view name).
3. **Определение представления:** DS передает логическое имя в **ViewResolver**. Тот подбирает конкретную технологию визуализации (например, Thymeleaf, JSP или FreeMarker) и возвращает объект View.
4. **Рендеринг:** DS передает данные модели в объект **View**, который формирует итоговый HTML-ответ и отправляет его обратно в браузер пользователя.

Вышеупомянутые компоненты входят в состав **WebApplicationContext**. Он является расширением стандартного **ApplicationContext** и дополнен функционалом, необходимым для работы в веб-среде (например, поддержкой областей видимости **Request** и **Session**, а также связью с **ServletContext**).

Контроллеры в Spring MVC

Аннотации `@Controller` и `@RestController` являются ключевыми компонентами модуля Spring MVC и используются для обработки входящих HTTP-запросов. Несмотря на схожее назначение, они отражают два разных подхода к разработке веб-приложений: классическую многостраничную архитектуру (МПА) и сервис-ориентированную

архитектуру (REST API). Выбор между ними определяет, будет ли сервер возвращать пользователю готовую веб-страницу или данные для дальнейшей обработки на клиенте.

Основное различие между `@Controller` и `@RestController` заключается в том, как обрабатывается возвращаемое значение методов. В классе с аннотацией `@Controller` методы по умолчанию возвращают имя представления (например, `String`), которое затем используется для рендеринга HTML-страницы. Если метод дополнительно помечен `@ResponseBody`, его результат записывается напрямую в тело HTTP-ответа и может иметь любой тип.

`@RestController` фактически объединяет аннотации `@Controller` и `@ResponseBody`. Поэтому все методы такого класса автоматически возвращают данные, которые записываются непосредственно в тело HTTP-ответа.

`@Controller` используется в рамках архитектуры MVC: контроллер взаимодействует с моделью, получает данные и передаёт их представлению для генерации HTML-страницы с помощью шаблонизаторов, например Thymeleaf или JSP.

`@RestController` применяется для создания REST-сервисов. Его методы возвращают объекты, которые автоматически сериализуются в формат ответа (чаще всего JSON, реже XML).

ObjectMapper

Преобразование Java-объектов в JSON в Spring выполняется автоматически с помощью библиотеки **Jackson**. При наличии зависимости `spring-web` или `spring-boot-starter-web` фреймворк регистрирует в контексте Spring бин `ObjectMapper` — основной класс Jackson, отвечающий за сериализацию и десериализацию объектов.

Когда метод контроллера возвращает объект (например, `List<...>`), Spring использует механизм `HttpMessageConverter`. Он выбирает подходящий конвертер — обычно `MappingJackson2HttpMessageConverter`, который с помощью `ObjectMapper` преобразует Java-объект в JSON и записывает результат в тело HTTP-ответа.

Поскольку `ObjectMapper` зарегистрирован как Spring-бин, его можно внедрять (`@Autowired`) и использовать напрямую — например, для ручной сериализации, десериализации или настройки поведения преобразования JSON.

Request Mapping

Ссылка на доку: <https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-requestmapping.html>

Пришло время рассмотреть аннотации, которые используются для маршрутизации и определения методов обработки HTTP-запросов.

```
@RestController
@RequestMapping("/api")
public class CourseController {
    private final CourseRepository courseRepository;

    public CourseController(CourseRepository courseRepository) {
        this.courseRepository = courseRepository;
    }

    @GetMapping("/courses")
    public List<Course> courseTable() {
        return courseRepository.findAll();
    }
}
```

Аннотация `@RequestMapping` определяет какой контроллер и какой метод будут обрабатывать запрос. При указании её над контроллером она определяет префикс для всех его эндпойнтов.

Также можно указать её над методом и указать путь и метод обрабатываемого запроса:

```
...
@RequestMapping(value="/courses", method=RequestMethod.GET)
public List<Course> courseTable() {
    return courseRepository.findAll();
}
```

Остальные аннотация для маппинга являются шорткатами для неё:

- `@GetMapping`
- `@PostMapping`
- `@PutMapping`
- `@DeleteMapping`
- `@PatchMapping`

Сокращения были добавлены, т.к. методам следует быть привязанными к конкретному HTTP-методу, `@RequestMapping`, по умолчанию, принимает любой. `@RequestMapping` все ещё рекомендуется использовать на уровне класса, чтобы указывать префиксы.

Посмотрим другие используемые аннотации:

```

...
@ResponseStatus(HttpStatus.CREATED)
@PostMapping("/articles/{id}/comments")
public Comment createComment(@RequestBody CommentCreateDTO dto,
                             @PathVariable String id) {

    // smth in the code
    // wooooo
}

```

`@RequestBody` применяется для десериализации тела запроса в объект с помощью [HttpMessageReader](#) .

`@ResponseStatus` явно задаёт статус ответа.

Аннотация `@PathVariable` в Spring MVC позволяет извлекать динамические значения из пути URL и связывать их с параметрами методов контроллера. Сами пути поддерживают следующие шаблоны:

Шаблон	Описание	Пример
spring	Точный путь	/spring -> /spring
?	1 любой символ	/pages/t?st.html -> /pages/test.html , /pages/t3st.html
*	0+ символов в одном сегменте пути	/resources/*.png -> /resources/file.png /projects/*/versions -> /projects/spring/versions
**	0+ сегментов пути	/resources/** -> /resources , /resources/img/file.png /**/info -> /info , /spring/info
{name}	Сегмент как переменная	/projects/{project}/versions -> project=spring
{name:regex}	Переменная с regex	/projects/{project:[a-z]+}/versions -> /projects/spring/versions
{*path}	Неск. сегментов как переменная	/resources/{*file} -> file=/images/file.png

А также есть:

- `@RequestParam` для параметров Query.
- `@RequestHeader` для HTTP-заголовка
- `@CookieValue` для чтения Cookie

Про аннотации для валидации можно прочитать в соответствующем пункте.

@ControllerAdvice . Обработка исключений.

Без централизованной обработки исключений:

- контроллеры засоряются try-catch
- ответы пользователю получаются непоследовательными
- сложно логировать и контролировать ошибки

Spring MVC имеет несколько механизмов для обобщения обработки исключений.

Аннотация `@ExceptionHandler` над методом внутри класса контроллера:

```
@RestController
@RequestMapping("/api")
public class CourseController {
    @GetMapping("/courses/{id}")
    public Course getCourse(@PathVariable("id") Long id) {
        return courseRepository.findById(id).orElseThrow();
    }

    // (В этом примере ApiError – некоторое кастомное DTO для исключений)
    @ExceptionHandler
    public ResponseEntity<ApiError>
    noSuchElementExceptionHandler(NoSuchElementException ex) {
        return new ResponseEntity<>(new ApiError(ex.getMessage()),
            HttpStatus.NOT_FOUND);
    }
}
```

Метод в примере выше будет вызван, если при выполнении какого-либо метода в данном контроллере возникнет исключение типа `NoSuchElementException` (параметр именно этого типа передается в метод `ExceptionHandler`). В данном методе нам нужно не только указать DTO, но и HTTP-статус ответа на запрос. Поэтому мы возвращаем экземпляр класса `ResponseEntity`.

Чтобы обрабатывать сразу несколько ошибок в одном хендлере, они указываются как параметр аннотации в фигурных скобках: `@ExceptionHandler({NullPointerException, NoSuchElementException})`

Чтобы не писать такие блоки для каждого контроллера, мы можем создать глобальный обработчик ошибок, используя аннотацию `@ControllerAdvice` / `@RestControllerAdvice` (разница между ними такая же, как у `@Controller` и

`@RestController` : "A shortcut annotation that combines `@ControllerAdvice` with `@ResponseBody` , in effect simply an `@ControllerAdvice` whose exception handler methods render to the response body.").

```
@RestControllerAdvice
public class HandleErrorService {

    @ExceptionHandler
    public ResponseEntity<ApiError>
noSuchElementExceptionHandler(NoSuchElementException ex) {
    return new ResponseEntity<>(
        new ApiError(ex.getMessage()),
        HttpStatus.NOT_FOUND
    );
}
}
```

Spring автоматически регистрирует классы с аннотацией `@ControllerAdvice` и применяет их ко всем контроллерам в вашем приложении. В частности, в нём можно написать глобальные хендлеры исключений. (А про другие способы применения идите сюда: <https://docs.spring.io/spring-framework/reference/web/webflux/controller/ann-advice.html>)

Также можно ограничить множество контроллеров, к которым применяется Advice, с помощью аннотаций:

```
// Target all Controllers annotated with @RestController
@ControllerAdvice(annotations = RestController.class)
public class ExampleAdvice1 {}

// Target all Controllers within specific packages
@ControllerAdvice("org.example.controllers")
public class ExampleAdvice2 {}

// Target all Controllers assignable to specific classes
@ControllerAdvice(assignableTypes = {ControllerInterface.class,
AbstractController.class})
public class ExampleAdvice3 {}
```

Интерсепторы

[TODO | Will write later]

Валидация объектов

Валидация используется для проверки корректности данных, полученных от клиента, перед их сохранением в базе данных.

Позволяет предотвратить сохранение некорректных данных (например, пустых полей формы) и излишние if/else блоки в коде.

Для подключения модуля валидации добавим зависимость в pom.xml :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Правила валидации задаются в классе DTO с помощью аннотаций, указывающих условия, которым должны соответствовать поля:

```
public class CourseRequestToUpdate {

    @NotBlank(message = "Course author has to be filled")
    private String author;

    @NotBlank(message = "Course title has to be filled")
    private String title;

    // getters, setters
}
```

Аннотации находятся в пакете javax.validation.constraints , вот основные из них:

- @NotNull – значение не должно быть null
- @NotBlank – строка не должна быть null и пустой
- @Email – строка должна содержать корректный email
- @Pattern – строка должна соответствовать регулярному выражению
- @Size – длина строки должна находиться в заданных пределах
- @Max – значение не должно быть больше указанного
- @Min – значение не должно быть меньше указанного

Аннотации @Valid , и @Validated используются для запуска процесса валидации при обработке запроса в Spring. Между ними есть несколько ключевых различий:

- Происхождение: @Valid – это стандартная аннотация из спецификации Java Bean Validation, также известной как JSR-303. Она не является специфичной для Spring и

может использоваться в любом Java-приложении. С другой стороны, `@Validated` – это специфическая для Spring аннотация, предоставляемая самим Spring.

- Функция: `@Valid` используется для проверки объекта метода или параметра в методе. Зачастую она используется, когда объект получен в HTTP-запросе, и вы хотите проверить поля этого объекта. `@Validated` используется для проверки параметров метода на Spring-бине. Зачастую она используется, когда метод компонента Spring имеет параметры, которые должны проходить валидацию.
- Группировка: Только `@Validated` поддерживает группировку ограничений. Это полезно, когда для одного и того же объекта при разных обстоятельствах требуются разные группы проверок.

```
...
@ResponseStatus(HttpStatus.CREATED)
@PostMapping("/articles/{id}/comments")
public Comment createComment(@RequestBody @Validated CommentCreateDTO dto,
                             @PathVariable String id) {

    // smth in the code
    // wooooo
}
```

12. OpenAPI. Использование OpenAPI в документирование REST API. Инструменты для генерации документации OpenAPI в Spring.

OpenAPI – это спецификация для описания HTTP API в машинно-читаемом формате (JSON или YAML). Она позволяет формально описать:

- доступные endpoints
- параметры запросов
- формат тела запроса и ответа
- коды ответов
- схемы используемых моделей данных
- методы аутентификации

Наличие стандартизированной спецификации позволяет автоматически генерировать документацию, клиентские библиотеки и инструменты тестирования.

OpenAPI описывает структуру REST API в виде спецификации. На основе этой спецификации могут работать различные инструменты, например:

- Swagger UI – генерирует интерактивную веб-документацию
- генераторы клиентских SDK
- инструменты тестирования API

Swagger UI отображает список endpoint-ов, их параметры и ответы, а также позволяет отправлять HTTP-запросы прямо из браузера.

В Spring Boot спецификация OpenAPI может генерироваться автоматически на основе контроллеров приложения.

Для этого используется библиотека `springdoc-openapi`.

После подключения библиотека сканирует контроллеры Spring (`@RestController` , `@RequestMapping`) и автоматически формирует спецификацию OpenAPI.

После запуска приложения становятся доступны следующие endpoints:

- `/v3/api-docs` – спецификация OpenAPI в формате JSON
- `/swagger-ui.html` – интерактивная документация Swagger UI

Для уточнения документации используются специальные аннотации:

- `@Operation` – описание endpoint
- `@Parameter` – описание параметров
- `@ApiResponse` – описание ответов
- `@Schema` – описание моделей данных

Эти аннотации позволяют более подробно описать API и улучшить автоматически генерируемую документацию.

13. Hibernate. Как Hibernate связан с JPA. Как работают `@OneToMany` , `@ManyToOne` , `@ManyToMany` ? В чем разница между EAGER и LAZY загрузкой? Жизненный цикл сущности в Hibernate. Транзакции в Spring (`@Transactional`), параметры `isolation` и `propagation` .

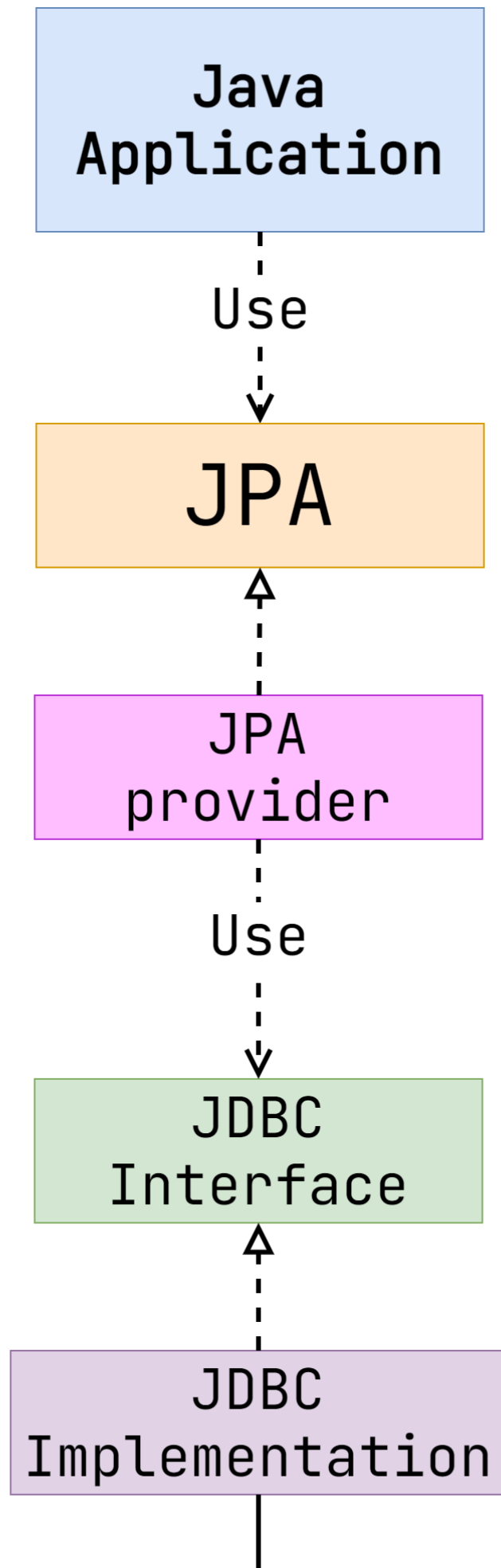
Hibernate. Как Hibernate связан с JPA.

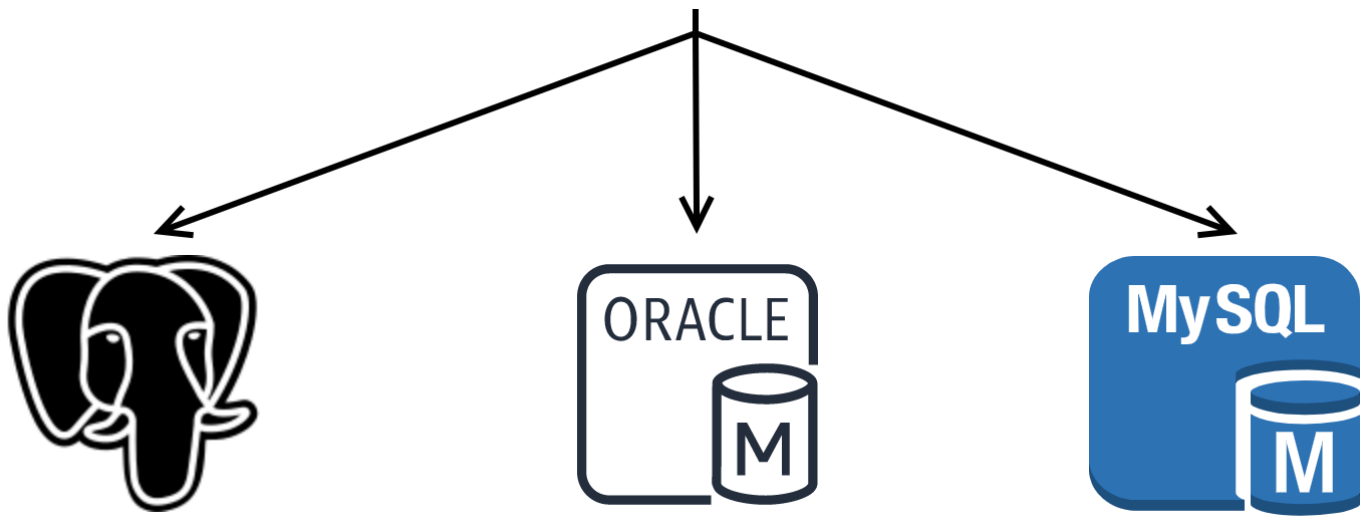
ORM (object-relation mapping) – это набор принципов и практик, которые позволяют соотнести таблицы в реляционной СУБД с объектами в Java-коде.

Основные концепции, характерные для почти любой ORM-библиотеки:

- Каждой таблице в БД соответствует Java-класс, именуемый сущностью (entity)
- Между полями класса-сущности и колонками таблицы устанавливаются однозначные соответствия таким образом, что данные из каждой строки таблицы могут быть записаны в экземпляр класса-сущности и наоборот
- Между сущностями возможны связи, аналогичные тем, что создаются в БД при помощи внешних ключей
- Существует специальный язык запросов, подобный SQL, но оперирующий не таблицами, а соответствующим им сущностям
- При выполнении запросов из Java-кода ORM-система преобразует их в SQL, а результаты переносит в коллекции классов-сущностей.

В мире Java эти принципы закреплены в стандарте **JPA (Jakarta Persistence API)**. Это общая спецификация, которая описывает правила и аннотации для работы с ORM, позволяя разработчикам использовать разные библиотеки по единому шаблону.





Hibernate – это Java-фреймворк, который реализует паттерн ORM. Он соответствует JPA-спецификации (т.к. появился раньше неё, и JPA был сформирован на основе идей Hibernate).

Spring Data JPA – это обертка для JPA, которая является частью Spring Framework. Также по умолчанию зависимость на Spring Data JPA тянет за собой и Hibernate. Но при желании можно заменить его на другую реализацию JPA.

Маппинг сущностей. `@OneToMany` , `@ManyToOne` , `@ManyToMany` . **EAGER** и **LAZY** загрузка.

Чтобы превратить класс в управляемую Spring Data и Hibernate сущность, нужно выполнить ряд требований:

- над классом сущности должна быть аннотация `@Entity`
- в классе сущности должно быть поле с уникальным идентификатором, над таким полем должна быть аннотация `@Id` (обозначает первичный ключ)
- у класса обязательно должен быть конструктор по умолчанию. Его лучше объявить как `protected` , чтобы его не мог использовать прочий бизнес-код.
- класс сущности не может быть inner-классом
- класс сущности не может быть `final` или `abstract` классом

Другие необязательные, но очень полезные аннотации:

- `@Table` указывает имя таблицы для сущности
- `@GeneratedValue` указывается над полем уникального идентификатора и определяет стратегию генерации ID. Например, `GenerationType.IDENTITY` рассчитывает, что БД сама будет генерировать ID при операции INSERT, а сам Hibernate не передаёт никакого значения в это поле.

- Над всеми полями, которые должно сохраниться в бд рекомендуется указать аннотацию `@Column`

```
@Entity
@Table(name = "courses")
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotNull(message = "Course author have to be filled")
    private String author;

    @NotNull(message = "Course title have to be filled")
    private String title;

    protected Course() {
    }

    public Course(String author, String title) {
        this.author = author;
        this.title = title;
    }

    // getters
}
```

Теперь разберёмся, как определять связи. Будем рассматривать три вида:

- `@OneToMany` — один ко многим
- `@ManyToOne` — многие к одному
- `@ManyToMany` — многие ко многим

`@ManyToOne` и `OneToMany`

Связь один ко многим (и многие к одному) в базе данных реализуются через внешний ключ. На практике чаще всего сначала описывается сторона с `@ManyToOne`, потому что именно она хранит этот ключ:

```
@Entity
@Table(name = "lessons")
public class Lesson {
```

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

private String title;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "course_id")
private Course course;

protected Lesson() {}
}

```

Аннотация `@JoinColumn` задаёт колонку внешнего ключа.

Связь можно дополнить обратной стороной:

```

@Entity
@Table(name = "courses")
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToMany(mappedBy = "course")
    private List<Lesson> lessons = new ArrayList<>();

    protected Course() {}
}

```

Поле `mappedBy` указывает, что связь уже описана в сущности `Lesson`.

Такая связь называется двусторонней, в ней:

- **owning side** – управляет связью (там `@JoinColumn`). Обычно Hibernate смотрит только на это поле при применении операций к БД.
- **inverse side** – только отражает связь (`mappedBy`), но через каскады можно сделать, чтобы изменения в коллекции влияли на сущности в базе (см. ниже).

Тип загрузки определяет, когда Hibernate будет подтягивать связанные данные.

- **LAZY** – данные загружаются только при обращении к полю
- **EAGER** – данные загружаются сразу вместе с сущностью

```
@ManyToOne(fetch = FetchType.LAZY)
```

По умолчанию:

- `@ManyToOne` – EAGER
- `@OneToMany` – LAZY

Важно: lazy-связи работают только внутри активной Hibernate-сессии (транзакции), так как для подгрузки данных требуется живое соединение с базой. Если обратиться к ленивому полю после закрытия сессии, возникнет `LazyInitializationException`.

Чтобы избежать этой ошибки, можно использовать следующие подходы:

- **Использование `@Transactional`** : Аннотируйте метод или весь класс этой аннотацией. Если `@Transactional` стоит над классом, каждый его публичный метод будет выполняться в контексте транзакции. Это гарантирует, что сессия останется открытой до завершения метода, позволяя безопасно обращаться к любым lazy-полям внутри него.
- **Метод `hibernate.initialize()`** : Если вам нужно подготовить объект для использования за пределами транзакции (например, для передачи в контроллер), вызовите `hibernate.initialize(entity.getLazyCollection())` внутри сервисного слоя. Это принудительно загрузит данные из базы "здесь и сейчас", пока сессия еще активна.

В связях `OneToMany` часто используются каскады – они позволяют автоматически применять операции к дочерним сущностям. Hibernate отслеживает изменения в коллекции и синхронизирует их с базой при завершении транзакции. Настроить это можно добавлением параметров в `@OneToMany`.

Параметр `cascade` определяет, какие именно действия (EntityManager операции) должны "пробрасываться" от родителя к потомкам. Например, если выставить `CascadeType.PERSIST`, то при сохранении автора в базу автоматически сохранятся и все его новые книги. Если выбрать `CascadeType.REMOVE`, то при удалении автора удалятся и все связанные с ним записи. Это избавляет от написания лишнего кода для ручного управления каждой зависимой сущностью и гарантирует, что связанные данные будут обновлены синхронно.

Параметр `orphanRemoval` (удаление "сирот") отвечает за автоматическую очистку базы данных от объектов, которые больше не имеют связи с родителем. Когда вы устанавливаете `orphanRemoval = true` и удаляете дочерний объект из коллекции родителя (например, `author.getBooks().remove(book)`), Hibernate понимает, что эта сущность больше не нужна, и физически удаляет её из таблицы. В отличие от обычного каскадного

удаления, которое срабатывает только при уничтожении самого родителя, этот механизм следит за актуальностью связей внутри коллекции.

@ManyToMany

Эта связь реализуется через вспомогательную промежуточную таблицу, которая хранит пары идентификаторов из обеих таблиц.

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToMany
    @JoinTable(
        name = "course_user",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses = new HashSet<>();

    protected User() {}
}
```

Здесь:

- Владеющая сторона (Owning Side): Это сущность, где прописана аннотация `@JoinTable`. Именно она "отвечает" за обновление записей в промежуточной таблице.
- Обратная сторона (Non-owning Side, Inverse Side): В сущности `Course` поле помечается параметром `mappedBy`, который указывает на имя поля в классе-владельце:

```
@ManyToMany(mappedBy = "courses")
private Set<User> users;
```

- Если в промежуточную таблицу нужно добавить дополнительные данные (например, дату записи на курс, оценку или статус оплаты), стандартная связь `@ManyToMany` не справится.

- В этом случае связь разбивается на две цепочки `One-to-Many <-> Many-to-One`, а промежуточная таблица превращается в полноценную сущность с собственными полями.

Тут важно подметить, что сущности Hibernate мутабельны, и `equals` / `hashCode` могут с ними некорректно работать. Чтобы такого не возникало, `equals/hashCode` нужно строить по ID. Но здесь тоже есть нюанс. По умолчанию ID null, а потом Hibernate его заполняет. То есть опять же значение нестабильное. В качестве альтернативы можно использовать другую колонку, которая гарантированно уникальна и заполняется при создании объекта. Но такие колонки есть не всегда, тогда для ID можно использовать такой подход:

```
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
        if (!(o instanceof User user)) {
            return false;
        }
        return id != null && id.equals(user.id);
    }

    @Override
    public int hashCode() {
        return User.class.hashCode();
    }
}
```

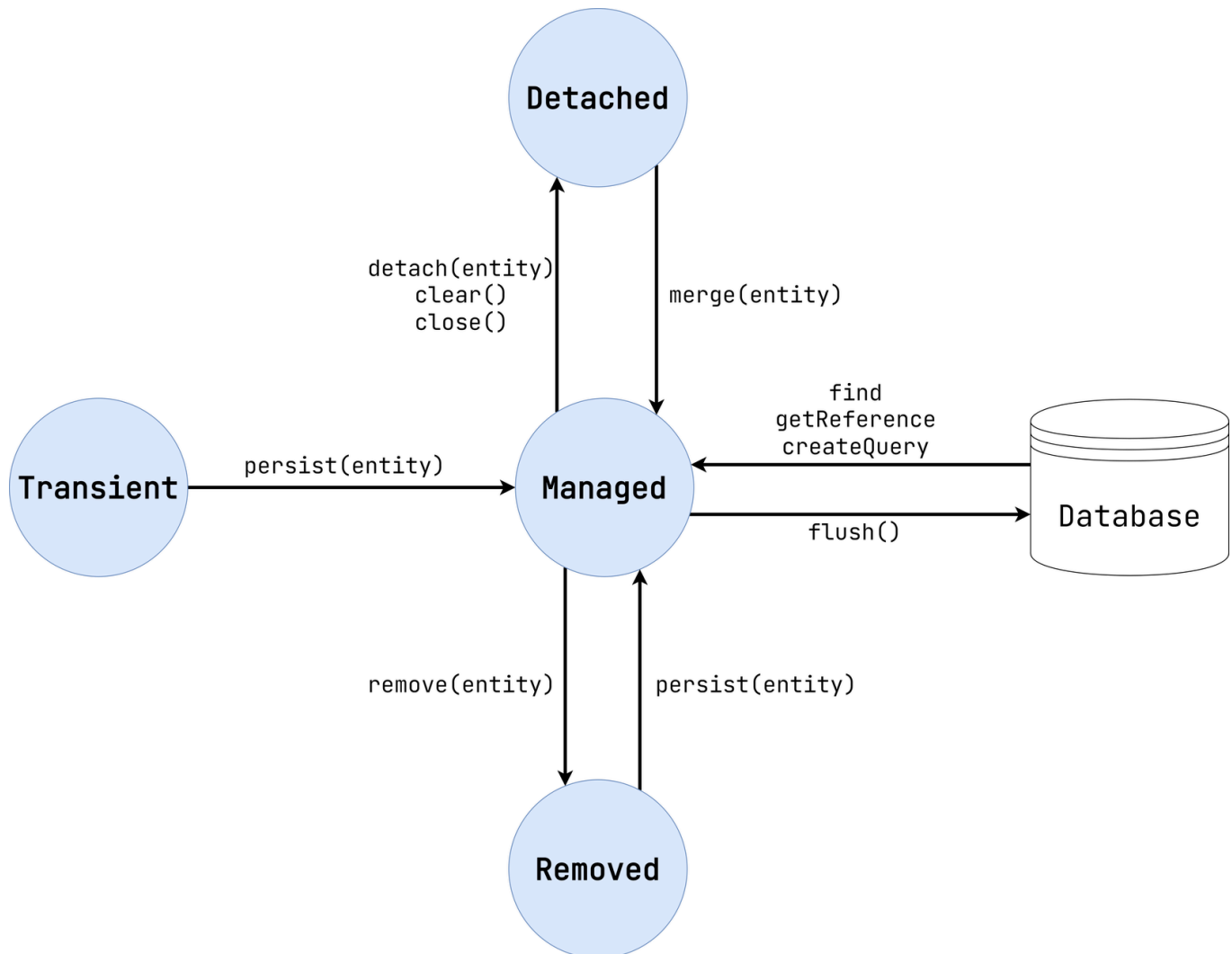
Логика здесь такая:

1. Функция `hashCode` всегда возвращает одинаковое значение. В данном случае, хэш от класса.
2. Если объекты равны по ссылкам в `equals`, возвращаем `true`.
3. Если переданный объект не является экземпляром или наследником `User`, возвращаем `false`.
4. Если `id` в текущем объекте равен `null`, мы не можем сказать, равен ли он переданной сущности или нет. Так что в этом случае мы всегда по умолчанию считаем объекты неравными и возвращаем `false`.

5. Иначе сравниваем id текущего и переданного объектов на равенство.

Жизненный цикл сущности в Hibernate

Под сущностью Hibernate мы понимаем класс с аннотацией `@Entity`, который маппится на таблицу в базе данных. Цикл жизни сущности состоит из 4 состояний: **Transient**, **Managed**, **Detached** и **Removed**.



Разберем каждое возможное состояние сущности по отдельности:

Состояние **Transient (New)** означает, что объект только что создан через конструктор и не отслеживается Hibernate. Даже если код выполняется внутри транзакции (`@Transactional`), никакие SQL-запросы (например, `insert`) не будут сгенерированы.

Состояние **Managed** – ключевое. В этом состоянии Hibernate отслеживает изменения полей сущности и автоматически преобразует их в SQL-запросы (`insert`, `update`, `delete`). Сущность становится Managed, если она сохранена через `persist` (или `save` в Spring Data JPA) либо загружена из базы, например через `findById`. Метод `save` внутри репозитория делегирует вызов либо `persist` (для новых сущностей), либо `merge` (для

уже существующих). При этом SQL-запросы не выполняются мгновенно – они накапливаются в Persistence Context и отправляются в базу при вызове `flush`, который обычно происходит автоматически перед коммитом транзакции. Важно понимать, что изменения Managed-сущности отслеживаются автоматически, поэтому вызов `save` после изменения необязателен, хотя и рекомендуется для читаемости кода.

Состояние **Removed** означает, что сущность помечена на удаление. После вызова `flush` Hibernate сгенерирует `delete` -запрос и удалит запись из базы данных.

Состояние **Detached** возникает, когда сущность ранее была Managed, но транзакция завершилась. В этом состоянии Hibernate больше не отслеживает изменения, поэтому любые модификации полей не приведут к SQL-запросам. При этом у сущности уже есть идентификатор, так как она ранее была сохранена. Это часто становится источником ошибок: разработчик изменяет объект и ожидает, что изменения сохранятся, но этого не происходит. Чтобы снова начать отслеживание, сущность нужно вернуть в состояние Managed, например через `merge` или вызов `save` в Spring Data JPA.

Транзакции в Spring (`@Transactional`), параметры `isolation` и `propagation` .

`@Transactional` – это аннотация Spring, реализованная через AOP, которая оборачивает метод в транзакционный контекст. Это означает, что все операции внутри метода выполняются как единое целое: либо полностью применяются (`commit`), либо полностью откатываются (`rollback`).

Spring создает прокси вокруг бина и управляет транзакцией на границе вызова метода. Важно, что транзакция работает только при вызове метода извне через прокси – внутренние вызовы методов внутри класса не проходят через `@Transactional` .

Базовое поведение:

- При успешном завершении метода происходит `commit`.
- При выбрасывании `RuntimeException` или `Error` происходит `rollback`.
- Checked exceptions по умолчанию не вызывают `rollback` (это можно изменить через `rollbackFor`).

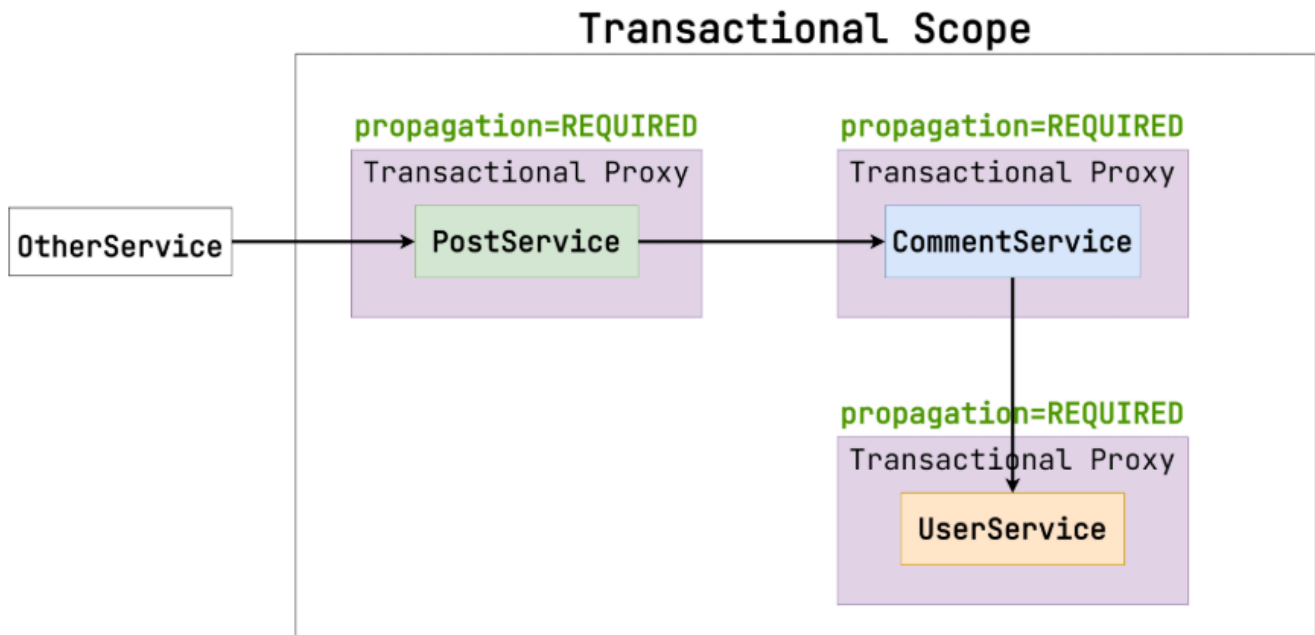
Основные параметры `@Transactional`

Параметр `propagation` определяет, как метод ведет себя относительно уже существующей транзакции.

По умолчанию: `REQUIRED`

- Если транзакция есть – используется она.

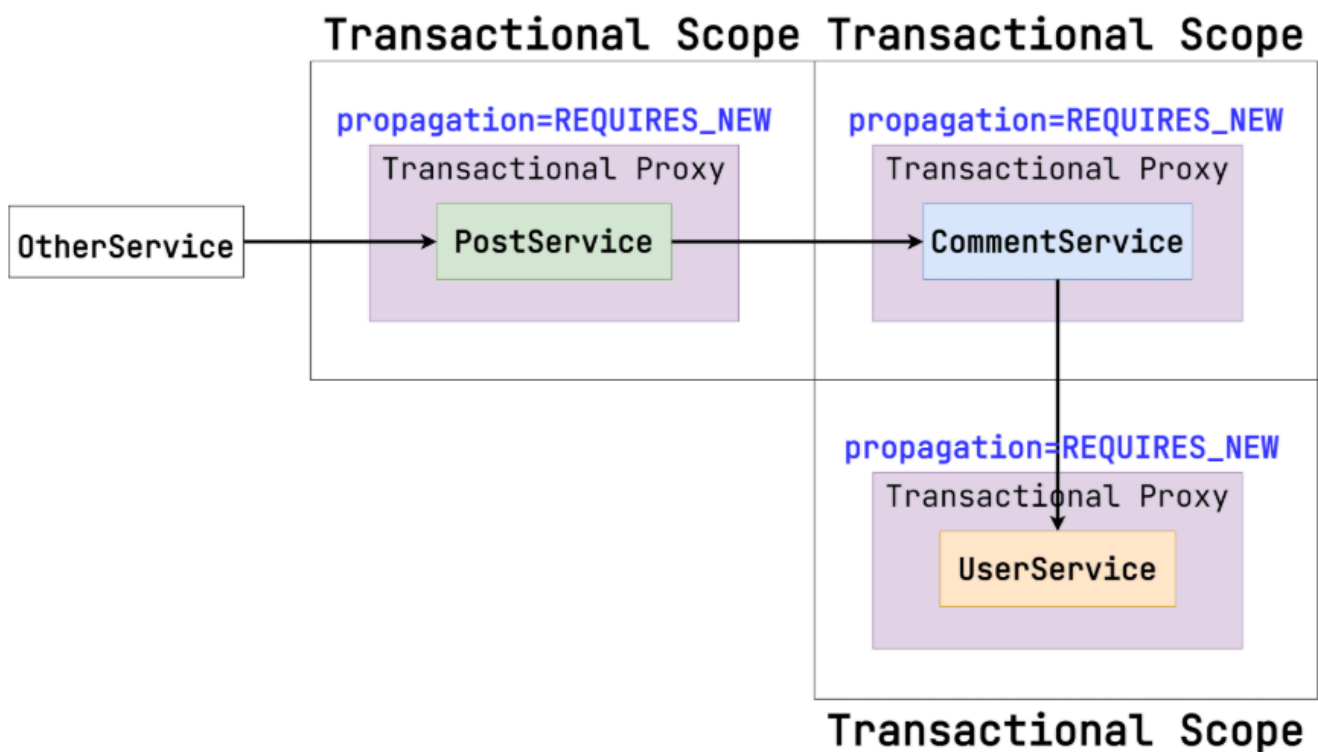
- Если нет – создается новая.



Это наиболее часто используемый режим. Все вызовы в цепочке работают в рамках одной транзакции.

REQUIRES_NEW всегда создает новую транзакцию.

- Если уже есть активная – она приостанавливается.
- После завершения новой транзакции – предыдущая продолжается.



Каждый уровень – отдельная транзакция.

Особенности:

- Требуется отдельного соединения с БД.
- Может приводить к неожиданному поведению при чтении данных:
 - Например, при `READ_COMMITTED` изменения из внешней транзакции не видны во внутренней, если она еще не закоммичена.
- Часто используется для логирования, аудита, отправки событий – когда важно сохранить данные независимо от основной операции.

В Spring есть и другие варианты `propagation`, но приведенные выше являются наиболее часто используемыми.

Параметр `isolation` определяет уровень изоляции транзакции – то, какие изменения других транзакций видны текущей.

По умолчанию: `DEFAULT` (использует уровень БД, например в PostgreSQL – `READ_COMMITTED`).

Основные уровни:

- **READ_UNCOMMITTED**
 - Можно читать незакоммиченные данные (dirty reads).
 - Почти не используется.
- **READ_COMMITTED** (частый дефолт)
 - Видны только закоммиченные данные.
 - Возможны non-repeatable reads.
- **REPEATABLE_READ**
 - Данные, прочитанные в транзакции, не изменятся до ее конца.
 - Возможны phantom reads.
- **SERIALIZABLE**
 - Максимальная изоляция.
 - Транзакции ведут себя как последовательные.
 - Самый безопасный, но самый медленный.

Выбор уровня – это баланс между консистентностью и производительностью.

Параметр `readOnly` – флаг, обозначающий, что транзакция предназначена только для чтения.

```
@Transactional(readOnly = true)
public List<User> getUsers() { ... }
```

При использовании Spring Data JPA он помогает пропустить этап Hibernate Dirty Checking перед коммитом, что улучшает быстродействие. Также визуально отделяет readonly операции от read-write.

Важно отметить, что readonly не связан с propagation. Он выставляется один раз при старте транзакции и более не проверяется.

14. Spring Security. Настройка аутентификации через базу данных. Реализация JWT-аутентификации в Spring Security.

[TODO]

15. Синхронное и асинхронное взаимодействие в распределенной среде, их преимущества и недостатки. Реализации инструментов (HTTP, брокеры сообщений и т.д).

Определение межсерверного взаимодействия

Под межсерверным взаимодействием мы понимаем обмен данными между узлами, расположенными на удаленных хостах. Его принято делить на два типа: синхронное и асинхронное.

Синхронное взаимодействие строится по принципу "запрос – ожидание":

1. Клиент отправляет запрос.
2. Система приостанавливает дальнейшие действия и ждет ответа от сервера.
3. После получения результата клиент продолжает работу.

Пока операция не завершена, процесс "заблокирован" – именно поэтому такое взаимодействие называют синхронным. Типичные примеры: протокол HTTP и вызовы через фреймворк gRPC.

Для **асинхронного взаимодействия** обычно используют посредника – **брокер сообщений** (например, RabbitMQ или Kafka). Это отдельный компонент системы, через который узлы общаются друг с другом, не зная о расположении соседа. Отправителя в такой схеме называют **producer**, а получателя – **consumer**. При этом один и тот же сервис может совмещать обе роли: отправлять задачи в одну очередь и в то же время читать события из другой.



Главное отличие от HTTP – отсутствие прямой блокировки. В синхронной модели клиент ждет ответа от сервера, пока тот выполняет операцию. В асинхронной же схеме **producer** ждет только подтверждения от брокера, что сообщение сохранено. Ему не важно, свободен ли сейчас **consumer** и сколько времени займет обработка. Это идеально подходит для тяжелых задач вроде генерации годовых отчетов: мы просто "забрасываем" задачу в брокер и продолжаем работу, а когда отчет будет готов, сервис-обработчик пришлет уведомление в ответную очередь.

Признаки того, что стоит выбрать синхронное взаимодействие:

- Операция выполняется быстро (до 1 секунды).
- Результат нужен здесь и сейчас. Отложить его нельзя, потому что без него нельзя продолжить дальнейшую работу.
- Снижение доступности является приемлемым (если сервер упадет, клиент без его ответа тоже не сможет работать корректно).

Признаки того, что выбрать асинхронное взаимодействие:

- Операция выполняется долго (1 минуту и более):
- Получение результата можно отложить (например, отправить его клиенту на почту через несколько минут). Нет необходимости ждать его сейчас.
- Снижение доступности неприемлемо: если consumer упал, producer все равно должен продолжить работу (сообщение в брокер попадет, но consumer прочитает его позже).

Преимущества и недостатки подходов

Несмотря на простоту, синхронный подход делает систему хрупкой: если один сервис в цепочке "зависает" или падает, по цепочке встают и все остальные. Это явление называют каскадным отказом. Кроме того, такая модель требует прямой связности между узлами – клиент должен точно знать адрес сервера и "достучаться" до него в момент

запроса. Чтобы минимизировать эти риски, в синхронную среду внедряют дополнительные паттерны устойчивости: таймауты (Timeout), повторные запросы (Retries), автоматическое размыкание цепи (Circuit Breaker), контроль уникальности операций через Idempotency Key и т.д.

Асинхронность же значительно повышает отказоустойчивость и позволяет системе справляться с пиковыми нагрузками (брокер выступает буфером), но взамен усложняет разработку. Становится труднее отслеживать путь сообщения, сложнее обрабатывать ошибки (например, если сообщение потерялось внутри брокера) и невозможно гарантировать, что данные будут обработаны мгновенно. Мы получаем гибкость, но платим за неё сложностью мониторинга и отладки.

Реализации инструментов (в SpringBoot)

Синхронное взаимодействие

RestTemplate

Для реализации синхронного межсервисного взаимодействия в Spring традиционно используется класс `RestTemplate` из модуля `spring-web`. Это блокирующий HTTP-клиент: поток, выполняющий запрос, будет ожидать ответа от удалённого сервиса.

Важно: на текущий момент `RestTemplate` находится в режиме поддержки (maintenance mode), и для новых проектов рекомендуется использовать `WebClient`.

Чтобы не нарушать принципы Dependency Injection, `RestTemplate` объявляется как bean в конфигурации:

```
@Configuration
public class RestTemplateConfiguration {
    @Bean
    public RestTemplate restTemplate(@Value("${university.service.base.url}")
String baseUrl) {
        return new RestTemplateBuilder()
            .rootUri(baseUrl)
            .build();
    }
}
```

Здесь `@Value` инжектит значение, прописанное в `application.properties`. Свойство `rootUri` задаёт базовый URL, который будет автоматически добавляться ко всем запросам.

Далее создадим Gateway-класс, инкапсулирующий взаимодействие с внешним сервисом:

```

@Service
public class UniversityGateway {
    private final RestTemplate restTemplate;

    // constructor

    public University createUniversity(String name) {
        ResponseEntity<University> universityResponse = restTemplate.postForEntity(
            "/api/universities?name={name}",
            null,
            University.class,
            Map.of("name", name)
        );
        return universityResponse.getBody();
    }
}

```

Метод `postForEntity` выполняет HTTP POST-запрос и возвращает `ResponseEntity`, содержащий статус, заголовки и тело ответа.

На практике также необходимо обрабатывать ошибки (например, ответы 4xx/5xx), используя `ResponseErrorHandler` или перехватывая исключения `HttpStatusCodeException`.

Для unit-тестов `RestTemplate` обычно мокируется, однако такие тесты не проверяют корректность HTTP-взаимодействия.

Для интеграционного тестирования используется, например, `MockServer`. Он позволяет эмулировать внешний HTTP-сервис и проверять, какие именно запросы отправляет приложение. В сочетании с `Testcontainers` можно поднять `MockServer` в Docker-контейнере и перенаправить на него все HTTP-запросы:

- переопределяется `base.url` на адрес контейнера
- настраиваются ожидаемые запросы и ответы
- проверяется поведение Gateway

Такой подход позволяет тестировать как успешные, так и ошибочные сценарии (например, ответы 4xx/5xx или некорректное тело ответа).

Timeout

Запросы по сети принципиально отличаются от локальных вызовов методов: время их выполнения заранее неизвестно и зависит от состояния сети и удалённого сервиса. Ответ может прийти за миллисекунды, а может значительно задержаться или не прийти вовсе (например, при сетевых проблемах или недоступности сервиса).

Чтобы избежать бесконечного ожидания, используется механизм **timeout** – максимальное время, в течение которого клиент ожидает выполнение операции. При его превышении выбрасывается исключение.

Пример настройки timeout для `RestTemplate` :

```
@Configuration
public class RestTemplateConfiguration {
    @Bean
    public RestTemplate restTemplate(
        @Value("${university.service.base.url}") String baseUrl,
        @Value("${university.service.timeout.seconds}") int secondsTimeout
    ) {
        Duration timeout = Duration.ofSeconds(secondsTimeout);
        return new RestTemplateBuilder()
            .setConnectTimeout(timeout)
            .setReadTimeout(timeout)
            .rootUri(baseUrl)
            .build();
    }
}
```

Поскольку `RestTemplate` использует Socket API, различают два типа timeout:

- **connect timeout** – максимальное время на установление TCP-соединения
- **read timeout** – максимальное время ожидания данных от сервера (между получением пакетов)

Корректная настройка timeout является важной частью обеспечения отказоустойчивости системы. Слишком большие значения могут приводить к "залипанию" потоков и деградации сервиса, а слишком маленькие – к ложным ошибкам при нормальной работе сети.

RateLimiter

Timeout решает проблему ожидания ответа на стороне клиента. Однако перегрузка может возникать и на стороне сервера – например, если он способен обрабатывать не более 100 запросов в секунду.

Для ограничения интенсивности запросов используется **Rate Limiter** – механизм, который ограничивает количество вызовов за заданный промежуток времени. При превышении лимита выполнение запрещается (обычно с выбросом исключения). Rate Limiter защищает как удалённый сервис от перегрузки, так и сам клиент от избыточного числа запросов.

Здесь и далее будем рассматривать готовые решения из модуля Resilience4j.

Пример использования:

```
@Service
public class UniversityGateway {
    private final RestTemplate restTemplate;

    // constructor

    @RateLimiter(name = "createUniversity", fallbackMethod = "fallbackRateLimiter")
    public University createUniversity(String name) {
        try {
            ResponseEntity<University> universityResponse =
restTemplate.postForEntity(
                "/api/universities?name={name}",
                null,
                University.class,
                Map.of("name", name)
            );
            return universityResponse.getBody();
        } catch (RestClientException e) {
            throw new CreateUniversityException("Error during university creation:
" + e.getMessage(), e);
        }
    }

    public University fallbackRateLimiter(String name, RequestNotPermitted e) {
        throw new CreateUniversityException(e.getMessage(), e);
    }
}
```

Аннотация `@RateLimiter` обрабатывается через AOP: Spring создаёт прокси-объект, который добавляет логику ограничения запросов при вызове метода.

Параметры:

- `name` — имя экземпляра Rate Limiter (используется в конфигурации)
- `fallbackMethod` — метод, который вызывается при срабатывании ограничения

Сигнатура `fallbackMethod` должна совпадать с исходным методом, а последним параметром передаётся исключение (`RequestNotPermitted`). В fallback-методе можно как пробросить ошибку, так и вернуть значение по умолчанию.

Конфигурация задаётся в `application.properties`:

```
resilience4j.ratelimiter.instances.createUniversity.limitForPeriod=100  
resilience4j.ratelimiter.instances.createUniversity.limitRefreshPeriod=1s  
resilience4j.ratelimiter.instances.createUniversity.timeoutDuration=1s
```

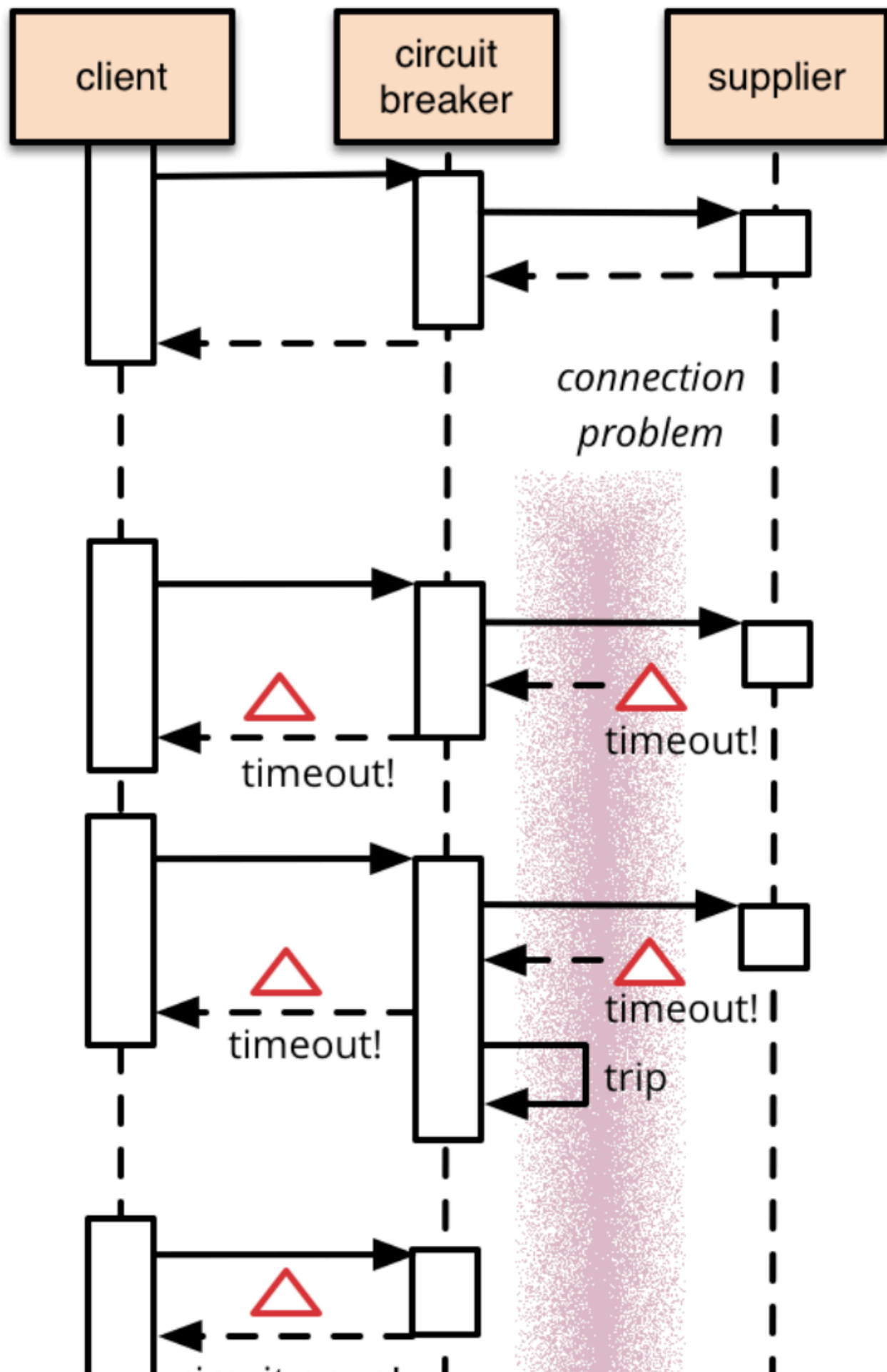
Параметры:

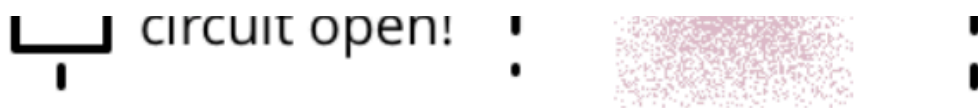
- `limitForPeriod` – максимальное количество вызовов за период
- `limitRefreshPeriod` – длительность периода, после которого счётчик сбрасывается
- `timeoutDuration` – максимальное время, в течение которого поток будет ждать "освобождения" лимита перед тем, как получить отказ

CircuitBreaker

Timeout и Rate Limiter ограничивают время ожидания и частоту запросов. Однако они не решают ситуацию, когда удалённый сервис стабильно недоступен (например, полностью упал). В этом случае клиент продолжает отправлять запросы, которые заведомо завершаются ошибкой.

Для решения этой проблемы используется паттерн **Circuit Breaker**.





Его идея – оборачивать вызов удалённого сервиса в "переключатель состояний", который может временно прекращать отправку запросов. Circuit Breaker имеет три состояния:

- **CLOSED** – нормальная работа, все запросы проходят
- **OPEN** – запросы не отправляются, сразу вызывается fallback
- **HALF_OPEN** – пробный режим: часть запросов отправляется для проверки восстановления сервиса

Переходы между состояниями:

- при превышении порога ошибок Circuit Breaker переходит из **CLOSED** в **OPEN**
- через заданное время – из **OPEN** в **HALF_OPEN**
- если пробные запросы успешны – **HALF_OPEN** в **CLOSED**, иначе снова **OPEN**

Пример использования:

```
@Service
public class UniversityGateway {
    private final RestTemplate restTemplate;

    // constructor

    @RateLimiter(name = "createUniversity", fallbackMethod = "fallbackRateLimiter")
    @CircuitBreaker(name = "createUniversity", fallbackMethod =
"fallbackCircuitBreaker")
    public University createUniversity(String name) {
        try {
            ResponseEntity<University> response =
                restTemplate.postForEntity(
                    "/api/universities?name={name}",
                    null,
                    University.class,
                    Map.of("name", name)
                );
            return response.getBody();
        } catch (RestClientException e) {
            throw new CreateUniversityException(
                "Error during university creation: " + e.getMessage(), e);
        }
    }
}
```

```

    public University fallbackRateLimiter(String name, RequestNotPermitted e) {
        throw new CreateUniversityException(e.getMessage(), e);
    }

    public University fallbackCircuitBreaker(String name, CallNotPermittedException
e) {
        throw new CreateUniversityException(e.getMessage(), e);
    }
}

```

Использование аналогично `RateLimiter`: аннотация обрабатывается через AOP, и Spring добавляет прокси с логикой Circuit Breaker.

Конфигурация:

```

resilience4j.circuitbreaker.instances.createUniversity.failureRateThreshold=10
resilience4j.circuitbreaker.instances.createUniversity.slowCallRateThreshold=10
resilience4j.circuitbreaker.instances.createUniversity.slowCallDurationThreshold=5000
resilience4j.circuitbreaker.instances.createUniversity.slidingWindowSize=100
resilience4j.circuitbreaker.instances.createUniversity.slidingWindowType=COUNT_BASED
resilience4j.circuitbreaker.instances.createUniversity.minimumNumberOfCalls=50
resilience4j.circuitbreaker.instances.createUniversity.waitDurationInOpenState=60s

```

Параметры:

- `failureRateThreshold` – процент ошибок, при превышении которого Circuit Breaker откроется
- `slowCallRateThreshold` – процент "медленных" вызовов, также влияющий на открытие
- `slowCallDurationThreshold` – порог времени (мс), после которого вызов считается медленным
- `slidingWindowSize` – размер окна для расчёта метрик
- `slidingWindowType`:
 - `COUNT_BASED` – последние N вызовов
 - `TIME_BASED` – последние N секунд
- `minimumNumberOfCalls` – минимальное количество вызовов перед началом анализа
- `waitDurationInOpenState` – время, которое Circuit Breaker остаётся в состоянии OPEN перед переходом в HALF_OPEN

Circuit Breaker предотвращает каскадные отказы (cascading failures), снижая нагрузку на недоступный сервис и ускоряя восстановление системы.

Retry и идемпотентность

При сетевом взаимодействии часто возникает неопределённость: клиент может получить **timeout**, хотя сервер уже обработал запрос.

Повторная отправка запроса (**retry**) в такой ситуации может привести к дублированию операции и некорректному состоянию системы.

Для безопасного использования retry необходимо обеспечить **идемпотентность** операции, т.е. что её повторное выполнение не изменяет итоговое состояние системы.

Рассмотрим следующую реализацию:

```
@Service
public class UniversityGateway {
    private final RestTemplate restTemplate;

    // constructor

    @RateLimiter(name = "createUniversity", fallbackMethod = "fallbackRateLimiter")
    @CircuitBreaker(name = "createUniversity", fallbackMethod =
"fallbackCircuitBreaker")
    @Retry(name = "createUniversity")
    public University createUniversity(String name, String requestId) {
        try {
            HttpHeaders headers = new HttpHeaders();
            headers.add("X-REQUEST-ID", requestId);

            ResponseEntity<University> response =
                restTemplate.exchange(
                    "/api/universities?name={name}",
                    HttpMethod.POST,
                    new HttpEntity<>(headers),
                    University.class,
                    Map.of("name", name)
                );

            return response.getBody();
        } catch (RestClientException e) {
            throw new CreateUniversityException(
                "Error during university creation: " + e.getMessage(), e);
        }
    }
}
```

```
// fallback methods...  
}
```

Конфигурация:

```
resilience4j.retry.instances.createUniversity.retry-  
exceptions[0]=com.example.CreateUniversityException  
resilience4j.retry.instances.createUniversity.wait-duration=500ms  
resilience4j.retry.instances.createUniversity.max-attempts=3
```

- `retry-exceptions` — исключения, при которых выполняется повтор
- `wait-duration` — задержка между попытками
- `max-attempts` — максимальное число попыток

Пояснение:

Аннотация `@Retry` повторно выполняет метод при возникновении заданных исключений.

Операции чтения и обновления по идентификатору, как правило, являются идемпотентными. Операции создания `POST` — нет, поэтому требуют дополнительной обработки. Для этого используется уникальный идентификатор запроса `request-id`:

- клиент генерирует уникальный идентификатор (например, `UUID`)
- передаёт его в каждом `retry` (например, в заголовке `X-REQUEST-ID`)
- сервер сохраняет результат выполнения по этому идентификатору
- при повторном запросе возвращает уже существующий результат

Важно заметить, что `requestId` должен быть одинаковым для всех повторных попыток: генерация идентификатора внутри метода, к которому применяется `@Retry`, приводит к тому, что каждая попытка будет иметь новый `requestId`, и дедупликация работать не будет.

Идентификатор должен передаваться извне — с уровня вызывающего кода.

На стороне сервера:

```
@RestController  
@RequestMapping("/api")  
public class UniversityService {  
  
    private final AtomicLong idGenerator = new AtomicLong(0);  
    private final Map<String, University> storage = new ConcurrentHashMap<>();  
  
    @PostMapping("/universities")
```

```

public University createUniversity(
    @RequestParam String name,
    @RequestHeader("X-REQUEST-ID") String requestId) {

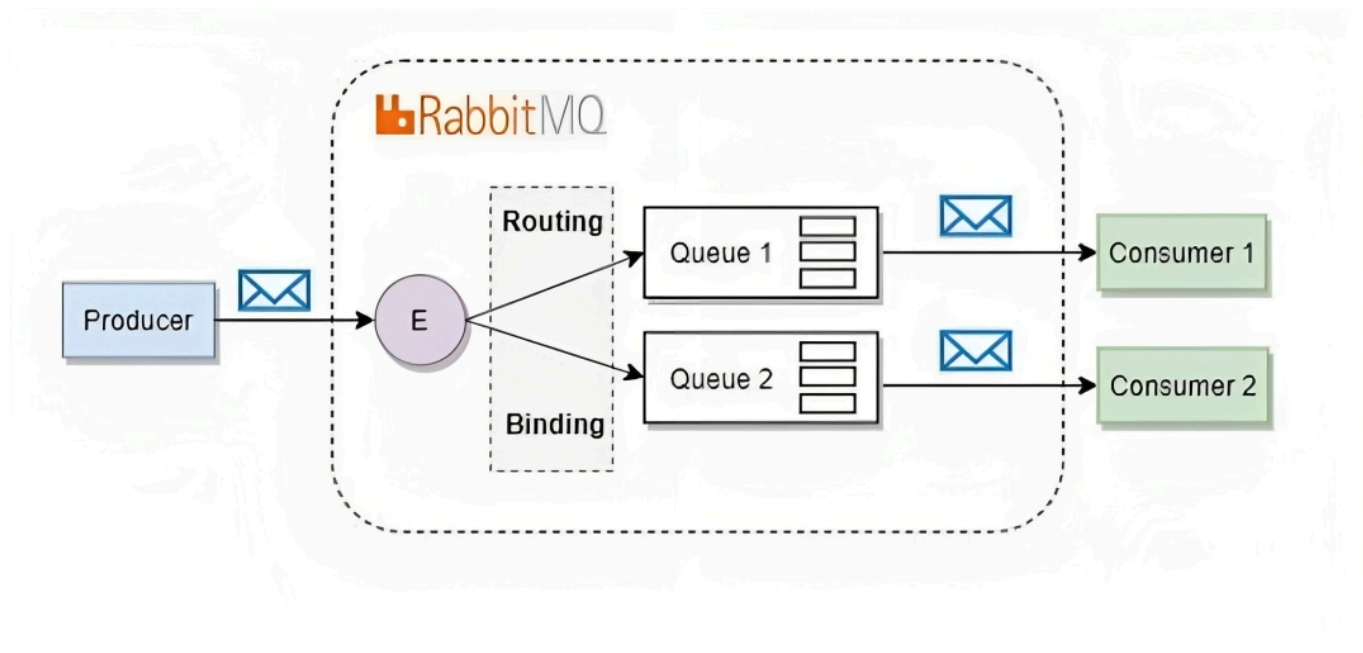
    return storage.computeIfAbsent(
        requestId,
        id -> new University(idGenerator.incrementAndGet(), name)
    );
}
}

```

Асинхронное взаимодействие

Перед рассмотрением Apache Kafka кратко отметим альтернативный подход на примере RabbitMQ.

RabbitMQ реализует модель обмена сообщениями на основе AMQP. Сообщения от продюсера сначала попадают в exchange, который на основе правил маршрутизации (bindings и routing key) направляет их в одну или несколько очередей.



Очереди выступают буфером, из которого сообщения считываются потребителями. После обработки потребитель отправляет подтверждение (**acknowledgment**), и только после этого сообщение удаляется из очереди. При отсутствии подтверждения сообщение может быть доставлено повторно.

Ключевые особенности RabbitMQ:

- маршрутизация сообщений на уровне брокера
- модель очередей (сообщение потребляется и удаляется)

- встроенная поддержка подтверждений доставки

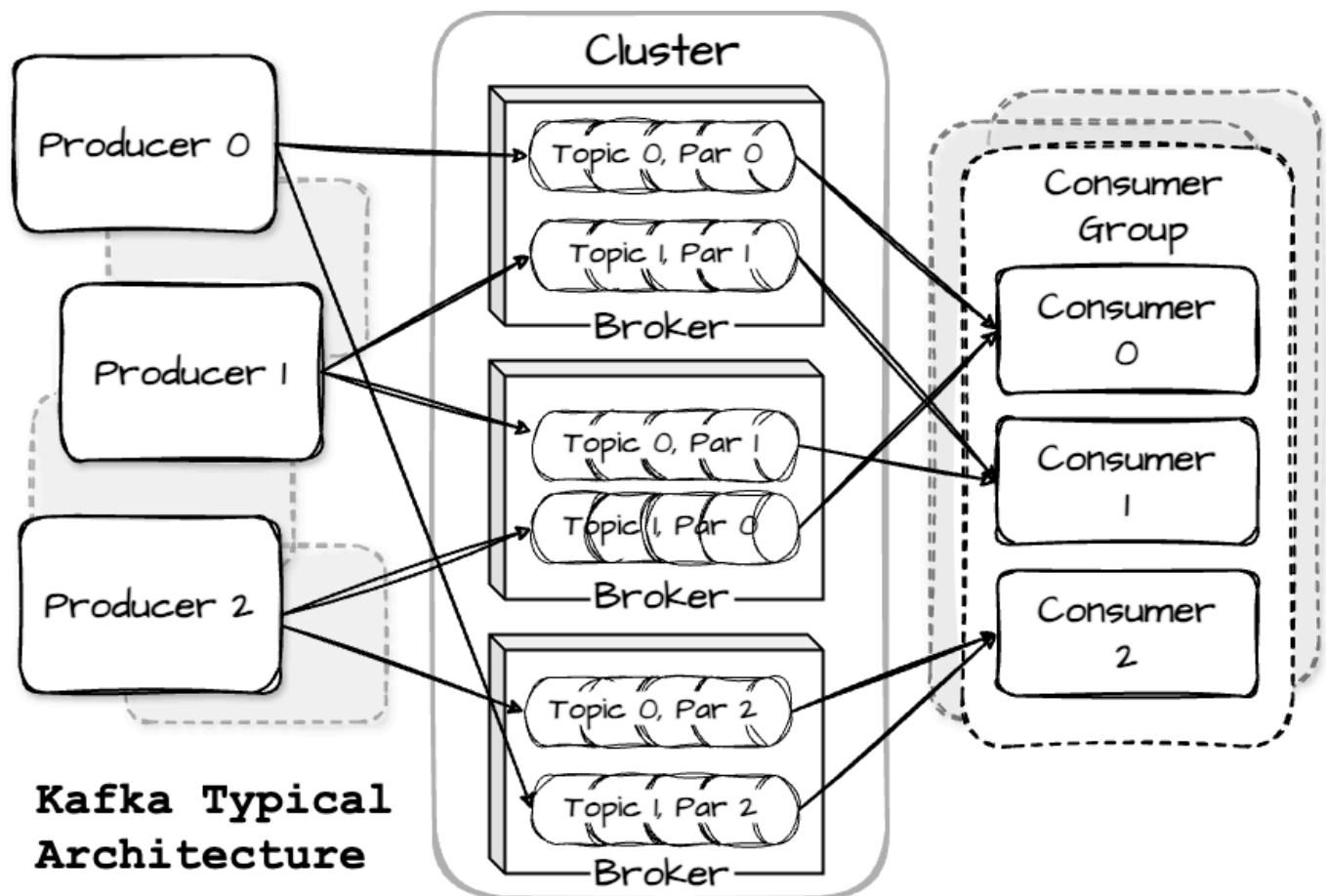
В отличие от этого, Apache Kafka использует другую модель (об этом подробнее ниже):

- сообщения записываются в лог (топик) и не удаляются сразу после чтения
- потребители читают данные независимо, управляя своим смещением (offset)
- один и тот же поток сообщений может быть прочитан многократно

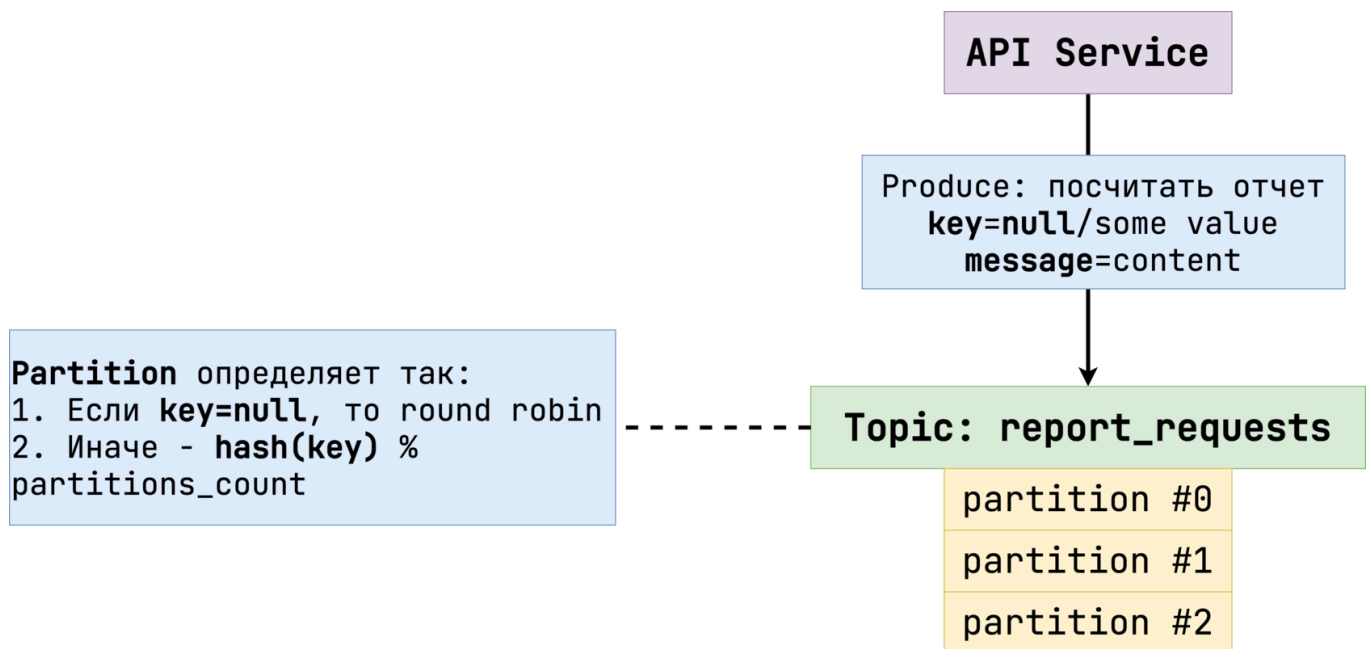
Таким образом, RabbitMQ ориентирован на распределение задач между потребителями, тогда как Kafka – на потоковую обработку и хранение событий.

Kafka

Apache Kafka – это распределённый брокер сообщений, основанный на модели журнала (log). Данные в Kafka организованы в **топики (topics)** – именованные потоки сообщений, в которые одни сервисы (producer) записывают события, а другие (consumer) их читают.



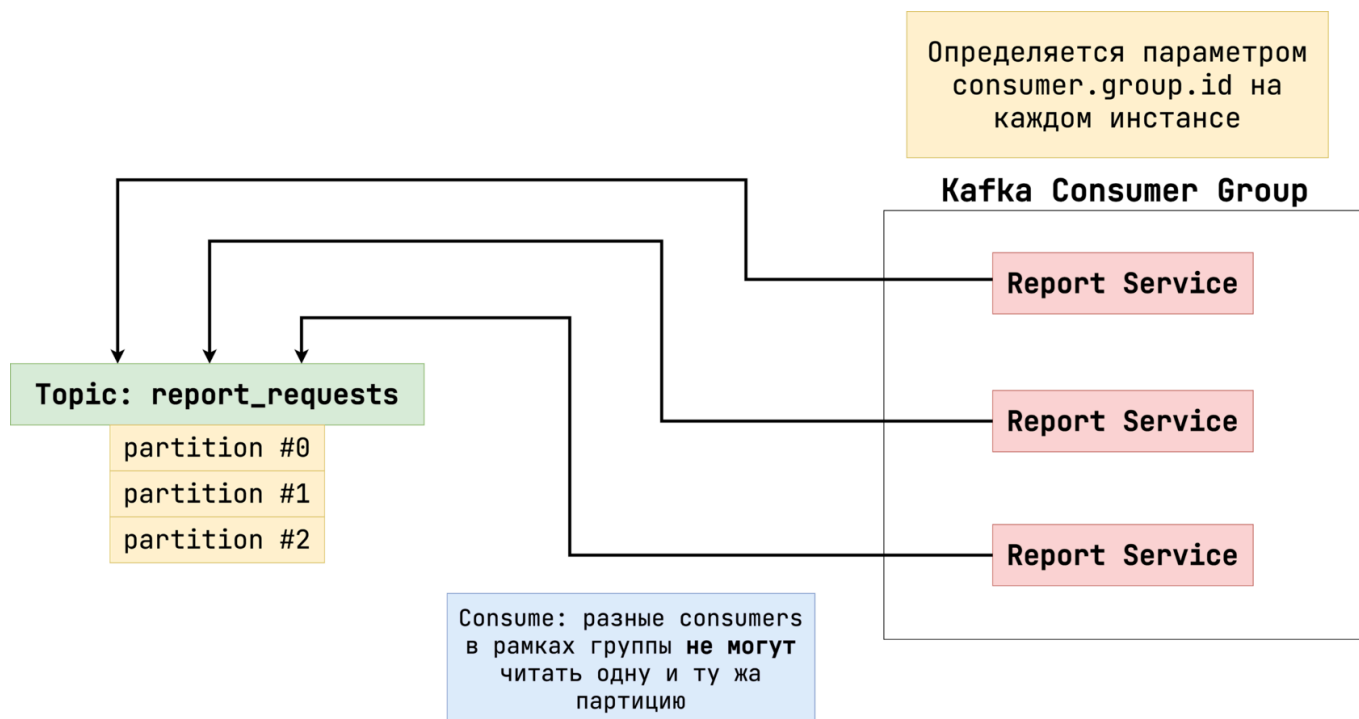
Каждый топик разбивается на **партиции (partitions)**. Партиция – это упорядоченная последовательность сообщений, которая физически хранится на брокерах. Именно партиции являются базовой единицей хранения и параллелизма. При отправке сообщения продюсер не пишет напрямую в "топик в целом", а выбирает конкретную партицию внутри него (см. схему ниже).



Выбор партии определяется следующим образом: если ключ сообщения не задан, используется равномерное распределение (round robin). Если ключ указан, применяется хеш-функция от ключа, и сообщение всегда попадает в одну и ту же партию. Это поведение важно не только для балансировки нагрузки, но и для сохранения порядка обработки.

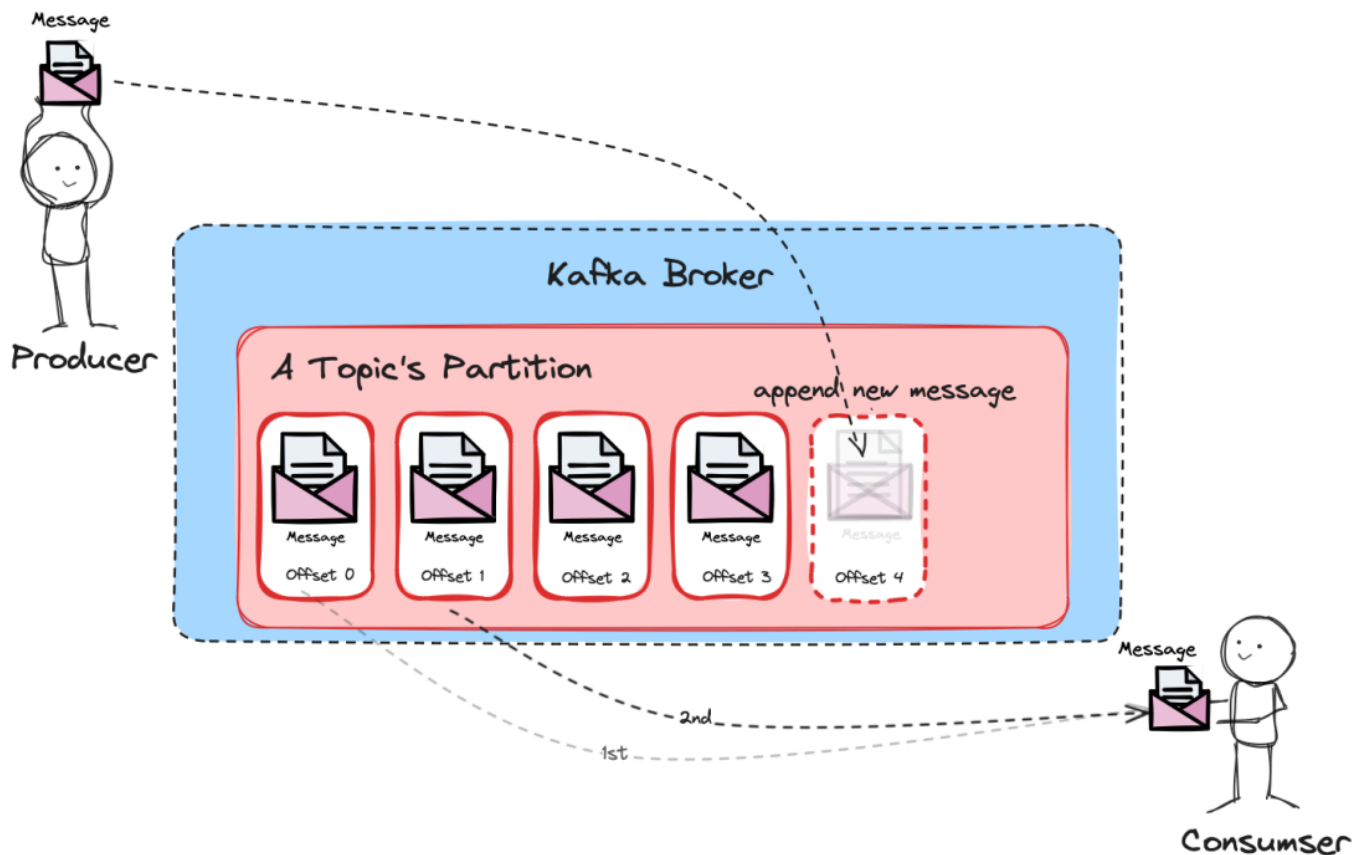
Порядок сообщений в Kafka гарантируется только внутри одной партии. Если связанные события (например, операции над одним и тем же объектом) попадут в разные партии, они могут быть обработаны в произвольном порядке. Поэтому для таких случаев используется ключ, который обеспечивает попадание связанных сообщений в одну партию и, как следствие, их последовательную обработку.

Чтение данных происходит через механизм **consumer group** (см. схему ниже). Несколько экземпляров сервиса могут объединяться в одну группу потребителей, и Kafka распределяет партии между ними. При этом каждая партия читается только одним consumer внутри группы, что исключает дублирование обработки. Таким образом достигается горизонтальное масштабирование: нагрузка делится между инстансами.



Количество партиций напрямую определяет максимальный уровень параллелизма. Если партиций меньше, чем потребителей, часть из них будет простаивать. Если больше – один consumer может обрабатывать несколько партиций. За счёт независимого чтения партиций Kafka обеспечивает высокую пропускную способность.

Внутри каждой партиции сообщения хранятся в виде непрерывного лога (см. схему ниже). Каждому сообщению присваивается **offset** – уникальный порядковый номер в рамках конкретной партиции. Новые сообщения всегда добавляются в конец лога. Потребители читают сообщения последовательно, начиная с определённого offset, и самостоятельно отслеживают, какие данные уже обработаны. Это позволяет в любой момент возобновить чтение с нужной позиции, например после перезапуска сервиса или при подключении новой consumer group.



Фиксация обработанных сообщений происходит через механизм commit. Потребитель сообщает брокеру, до какого offset данные считаются обработанными. Это можно делать автоматически или вручную. Ручное управление позволяет подтверждать обработку только после успешного выполнения бизнес-логики, что снижает риск потери данных или повторной обработки.

В отличие от классических очередей, Kafka не удаляет сообщения сразу после чтения. Они хранятся в течение заданного времени и могут быть прочитаны повторно другими consumer group. Это делает Kafka удобным инструментом для построения событийных систем, где важно не только обработать данные, но и иметь возможность воспроизвести их историю.

Отправка сообщений продюсером обычно происходит асинхронно: данные сначала накапливаются в буфере и затем отправляются батчами. Это повышает производительность, но требует дополнительного контроля в сценариях, где важна гарантия доставки. В таких случаях продюсер может дожидаться подтверждения от брокера, прежде чем считать операцию завершённой.

Пример использования в Spring

Для работы с Kafka в Spring Boot достаточно настроить подключение, отправку сообщений (producer) и их обработку (consumer).

Добавим следующую зависимость в сервисы:

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

Конфигурация:

Producer (course-service):

```
spring.kafka.bootstrap-servers=localhost:29093
topic-to-send-message=license-validation-check-request
```

Consumer (license-validation-service):

```
spring.kafka.bootstrap-servers=localhost:29093
spring.kafka.consumer.group-id=license-validation-service-group
spring.kafka.consumer.enable-auto-commit=true
topic-to-consume-message=license-validation-check-request
```

- `bootstrap-servers` – адрес Kafka-брокера
- `group-id` – идентификатор consumer group
- `enable-auto-commit` – автоматическая фиксация offset
- `topic-*` – топики для отправки/чтения сообщений

Пример Producer:

```
@RestController
@RequestMapping("/api")
@Validated
public class LicenseController {

    private final KafkaTemplate<String, String> kafkaTemplate;
    private final ObjectMapper objectMapper;
    private final String topic;

    public LicenseController(KafkaTemplate<String, String> kafkaTemplate,
                            ObjectMapper objectMapper,
                            @Value("${topic-to-send-message}") String topic) {
        this.kafkaTemplate = kafkaTemplate;
        this.objectMapper = objectMapper;
        this.topic = topic;
    }
}
```

```

    }

    @PostMapping("/courses/{courseId}:checkLicense")
    public void checkLicense(@PathVariable Long courseId) {
        String message = objectMapper.writeValueAsString(
            new ValidateCourseMessage(courseId)
        );

        kafkaTemplate.send(topic, message);
    }
}

```

Что происходит:

- формируется сообщение
- отправляется в Kafka-топик
- отправка происходит асинхронно (через внутренний буфер)

Пример Consumer:

```

@Service
public class LicenseValidateConsumer {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(LicenseValidateConsumer.class);

    private final ObjectMapper objectMapper;

    public LicenseValidateConsumer(ObjectMapper objectMapper) {
        this.objectMapper = objectMapper;
    }

    @KafkaListener(topics = {"${topic-to-consume-message}"})
    public void process(String message) {
        ValidateCourseMessage parsed =
            objectMapper.readValue(message, ValidateCourseMessage.class);

        LOGGER.info("Received message {}", parsed);
    }
}

```

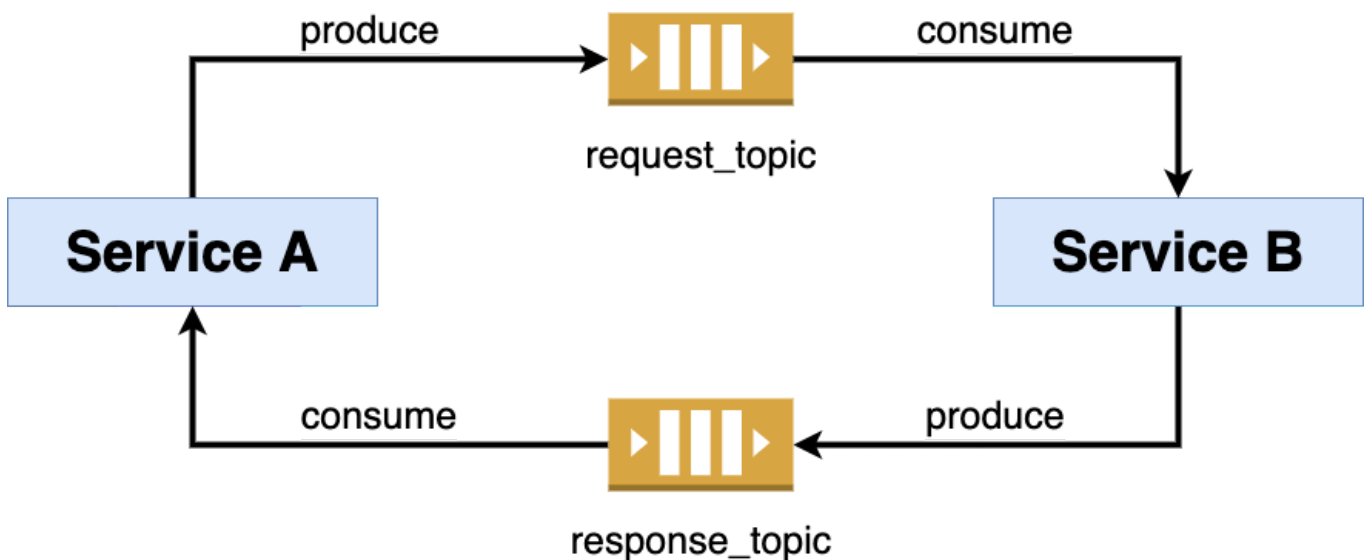
Что происходит:

- Spring создаёт consumer и подписывает его на топик

- под капотом используется polling (аналог `while(true) { poll() }`)
- при получении сообщения вызывается метод с `@KafkaListener`
- сообщение десериализуется и обрабатывается

Transactional Outbox и Saga Pattern

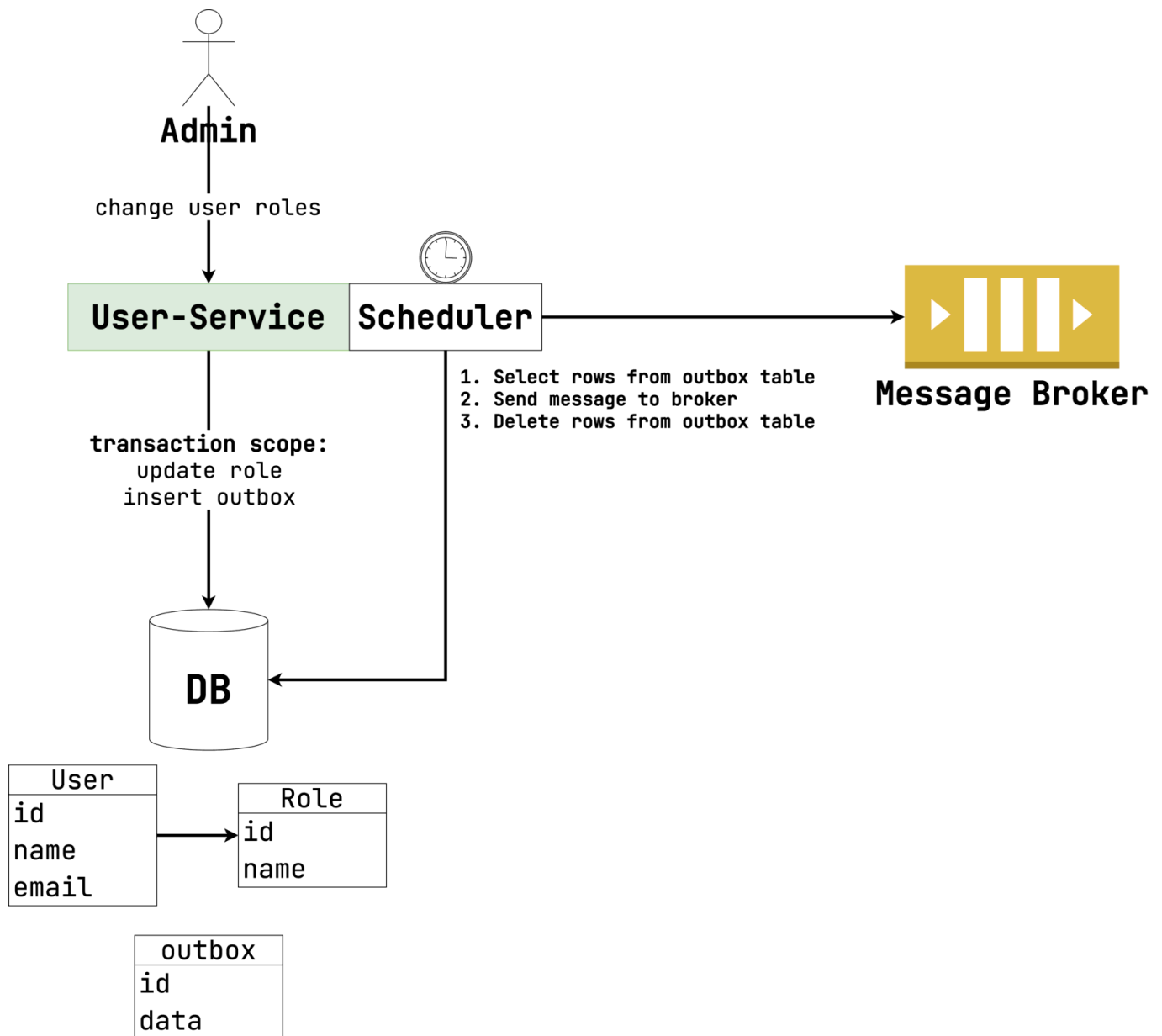
В синхронном HTTP-взаимодействии используется модель "запрос-ответ": клиент отправляет запрос и ожидает результат. Это упрощает логику, так как итог операции известен сразу. В асинхронной модели с брокером сообщений поведение иное: producer получает подтверждение только факта доставки сообщения в брокер, а обработка выполняется consumer независимо. При необходимости поведение "запрос-ответ" можно реализовать поверх брокера с помощью отдельных топиков.



Для этого используются два топика: `request` и `response`. Один сервис отправляет запрос в `request`, другой обрабатывает его и публикует результат в `response`. Инициатор читает ответ и завершает обработку. В такой схеме оба сервиса выступают одновременно и как producer, и как consumer.

При переходе к асинхронному взаимодействию возникает проблема консистентности между базой данных и брокером сообщений. В рамках одной бизнес-операции невозможно атомарно выполнить запись в БД и отправку события в Kafka. Это приводит к двум типам ошибок: либо событие отправлено без фактических изменений в БД, либо изменения зафиксированы, но событие потеряно.

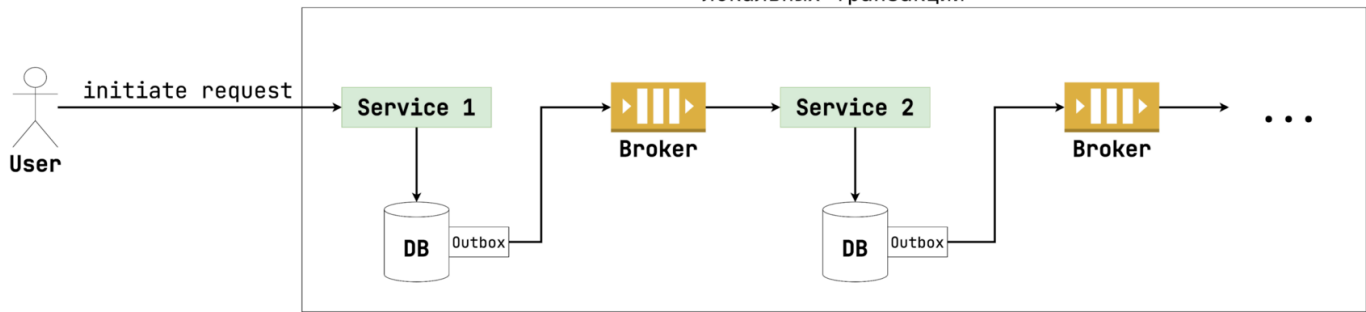
Для решения этой проблемы используется **Transactional Outbox Pattern**. Его идея – разделить процесс на два этапа. Сначала, в рамках одной транзакции с БД, выполняется бизнес-логика и сохраняется событие в таблицу outbox. Это гарантирует, что изменения и событие зафиксированы вместе. Затем отдельный процесс читает записи из outbox и отправляет их в Kafka, помечая как обработанные. Такой подход изолирует работу с брокером от основной транзакции и повышает устойчивость системы к сбоям.



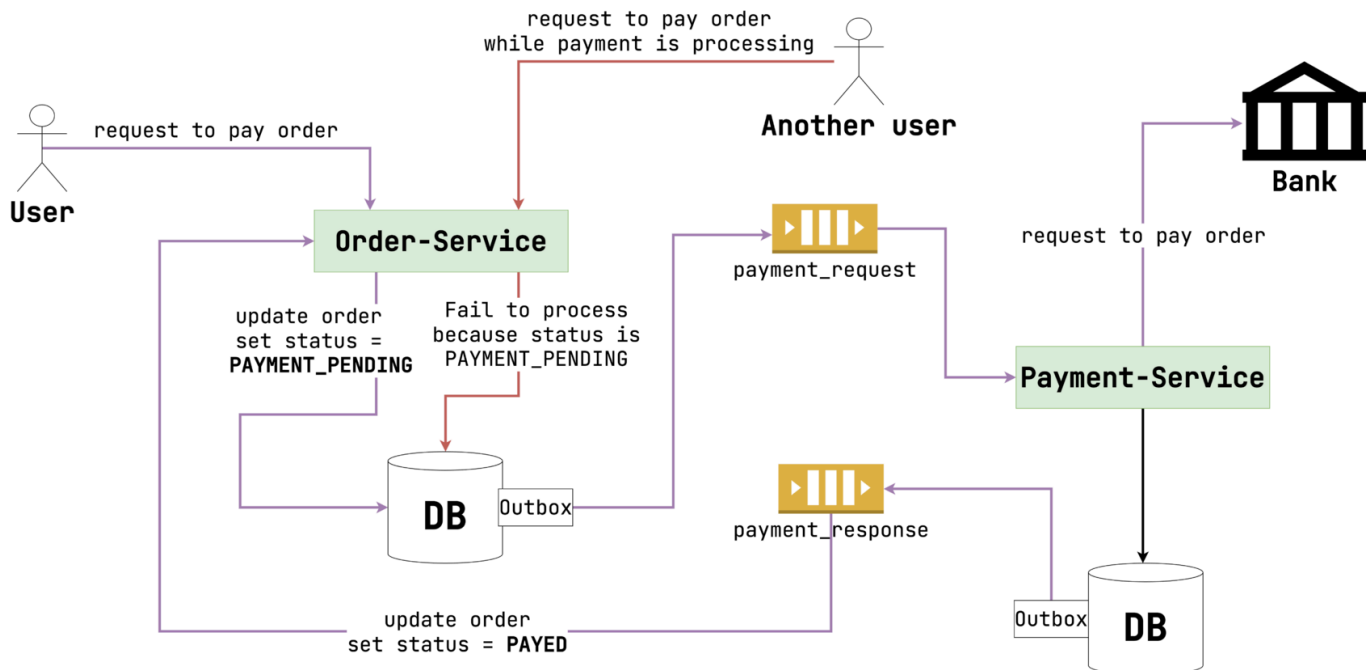
При этом важно учитывать модель доставки Kafka – *at least once*. В случае сбоев одно и то же сообщение может быть отправлено повторно, поэтому потребители должны быть идемпотентными. На практике это достигается, например, с помощью **Idempotency Key**, позволяющего безопасно обрабатывать дубликаты и приближаться к семантике *exactly once* на уровне бизнес-логики.

Однако сам по себе outbox решает лишь задачу надежной доставки событий. Для координации бизнес-операций между несколькими микросервисами используется паттерн **Saga**. Он объединяет последовательность локальных транзакций в разных сервисах в одну бизнес-операцию, где каждый шаг выполняется независимо, а взаимодействие происходит через события.

Одна бизнес-операция (например, оплата заказа) состоит из цепочки отдельных локальных транзакций



Saga по своей природе асинхронна: после запуска операция считается принятой в обработку, а ее завершение происходит позже. Это означает работу в модели eventual consistency. В таких условиях важно учитывать конкурентный доступ. Например, чтобы избежать повторной оплаты заказа, используется **semantic locking** – введение статуса (например, `PAYMENT_PENDING`), который запрещает повторное выполнение операции.



Поскольку глобальной транзакции нет, обработка ошибок строится через компенсирующие действия. Если один из шагов завершается неуспешно, выполняются обратные операции, возвращающие систему в согласованное состояние.

Координация шагов Saga может быть реализована двумя способами. В **хореографии** сервисы реагируют на события друг друга без центрального координатора, что снижает связанность, но усложняет контроль. В **оркестрации** один сервис управляет всей последовательностью шагов и хранит состояние операции, упрощая понимание и сопровождение ценой усложнения самого координатора.